

Social Engagement Through Interactive Design

An iterative approach to event discovery

Benjamin Honoré Nielsen, Cecilie Ståde Kristensen, Elia Hamann Ullerichs,
Luna Annabell Linné Jensen, Sebastian Thomsen, Sofie Louise Fuglsang

DAD8, Group 3, 2025 - 05

P8 – Project





Title:

Social engagement through interactive design

Theme:

Interaction Design and its process

Project Group:

DAD8 – Group 3

Group members:

Benjamin Honoré Nielsen
Study No. 20214136

Cecilie Ståde Kristensen
Study No. 20203139

Elia Hamann Ullerichs
Study No. 20212834

Luna Annabell Linné Jensen
Study No. 20213072

Sebastian Thomsen
Study No. 20214728

Sofie Louise Fuglsang
Study No. 20212701

Supervisor: Ashna Zada

Institute : Institute of computer science

Date of Completion: May 28, 2025

Pages: 92

Word count: 25.850

Appendix: 4

Abstract

In this project, we aim to showcase how an interactive design process - utilising HCI elements, user-centered participatory design approaches, and engaging with real potential users - can lead to a design solution in the shape of a prototype application. This application, created in android studio, coded in java, connects newcomers to their interests allowing them to participate in local events.

Contents

Preface.....	2
Generative AI as a tool	2
1 Introduction.....	3
2 Problem Domain	4
2.1 State of the Art.....	4
2.1.1 Human Relations.....	4
2.1.2 Community and Mental Health.....	6
2.1.3 The Importance of Places and Technology.....	8
2.1.4 Social Support and Resources at AAU and AU.....	9
2.1.5 Integration as Newcomers.....	10
2.1.6 Key take-aways.....	11
2.2 Problem Statement.....	12
2.3 Methods.....	12
2.3.1 Our HCI design process	12
2.3.2 Participatory Design Approach.....	14
2.3.3 Brainstorm.....	15
2.3.4 The 5W1H.....	17
2.3.5 The Open Group Interview	19
2.3.6 Workshop.....	19
3 Application Domain.....	22
3.1 Concept Development.....	22
3.2 Data Analysis.....	22
3.2.1 MoSCoW Requirements	24
3.2.2 Personas and Characters	27
3.2.3 First Iteration of the Prototype.....	30
3.2.4 Second Iteration of the Prototype	35

3.2.5	Changes in the Second Iteration Prototype.....	44
3.3	System Development	51
3.3.1	Object-Oriented Programming.....	51
3.3.2	Design of our application.....	52
4	Implementation of our Prototype	60
4.1	Which parts of our design were implemented?.....	60
4.2	Applications in Android.....	63
4.2.1	Project Structure and File Organisation	63
4.3	Code explanations.....	64
4.3.1	Implementation of the characters.....	65
4.3.2	Showing the character's description	68
4.3.3	Map	70
4.3.4	Timeline Of Events	70
4.3.5	Event Bubbles and Timeline Display	71
4.3.6	Making the calendar.....	77
4.3.7	Using RecyclerView	81
5	Evaluation of Prototype	84
6	Discussion	88
6.1	Usability Testing.....	89
7	Future Work.....	92
7.1	Integration with external calendars	92
7.2	Implementation of a ratings and feedback systems	92
8	Conclusion	94
9	Bibliography	96
10	List of Figures	99

Preface

We present this project, offering an in-depth look at the iterative process from idea to functional prototype. We would like to thank our project supervisor, Ashna, for her support and valuable feedback. We also thank all participants who contributed their time and insights during the workshop and usability test.

Generative AI as a tool

Throughout our project, generative AI was used as a supportive tool for various tasks, such as making grammatical corrections and assisting in formulating well-structured sections. AI-generated text was not included in the final report but served as a resource for identifying mistakes and providing inspiration when needed. During the early design phase, generative AI was also used to quickly create visual representations of our personas, which allowed efficient character design. While certain visual elements have inspired our final design, the artwork included in the project was drawn digitally by us in Clip Studio Paint. Additionally, generative AI supported the development process in coding and debugging, which helped with several aspects of the implementation. Using generative AI this way has helped us better understand the creative and technical processes involved, allowing us to further develop our skills in writing, designing, and programming an interactive system.

1 Introduction

Moving to a new city and adapting to unfamiliar surroundings can be socially challenging, especially for students who are unfamiliar with the environment and lack a pre-existing social network in the new environment. In Aalborg, newcomers often face challenges when it comes to building new relationships, often because they lack accessibility and relevant social opportunities. While platforms like StudieBobler and Genlyd offer students various opportunities in community building, a lot of the established social culture still centres around informal interactions and events. These types of interactions and events do not always match the individuals' personal interest or foster meaningful connections. This project aims to explore how a design solution can help newcomers discover local events and venues that align with their personal preferences, with the possibility of forming human relations. Rather than encouraging social interactions on its own, the goal is to guide newcomers towards experiences that match their interests while encouraging newcomers to participate in shared interests and activities. Through a participatory design process based in *Human-Computer Interaction* (HCI) and the *Design Thinking* model, we involved a combination of desk research, interviews, a workshop, brainstorming, and prototyping. The gathered insights laid the foundation for identifying the users' needs and requirements. This led to a concept developed with the users, featuring characters who guide the users towards events and social spaces which match their interests.

2 Problem Domain

2.1 State of the Art

In this section we go over the existing research and theories most relevant to the part of our goal of understanding the problem domain is to develop a clear definition of what context our eventual system should be able to administer (Mathiassen et al., 2018). We explore key concepts surrounding student well-being and integration by going over key concepts such as human relations, mental health and loneliness, the role of physical and digital spaces, and preexisting institutional solutions. These perspectives inform how we understand the different factors influencing a new student's ability to form social relations and lay the foundation for the subsequent problem formulation.

2.1.1 Human Relations

The significance of human relations and their positive and negative effects on students are essential topics in the problem domain. Understanding what constitutes a human relation and its various aspects, like community, communication, and relationship management, is crucial to grasp when understanding what constitutes as a socially healthy environment amongst students.

We must consider the role of social interaction in forming human relations. As human beings, we are embedded within an extensive network of interactions, both in real life through connections with friends, family, and acquaintances and in virtual spaces. The lives of individuals are shaped by the extent to which they are integrated into this broader web of human relations. In this discussion, we will explore the different aspects of human relations, drawing upon the book *Human Relationships* by Steve Duck (2007). We keep in mind that the author's point in this book is from 2007 at most, and as technology has developed a lot in the decades since, we apply his observations critically to a somewhat different technological landscape.

Interpersonal relations are generally understood as connections between two individuals who share common interests, experience mutual affection, or develop a strong bond. However, social relations extend beyond these conventional notions. At their core, interactions between individuals involve biological, psychological, and sociological mechanisms. Relations are not solely determined by mutual liking or disliking but rather by the connections that are formed

through conversation, the interpretation of body language, and underlying psychological and biological processes (such as hormonal influences). These elements collectively shape interpersonal connections.

A fundamental component of relations is communication. This term is used in its broadest sense, encompassing not only verbal interaction but also nonverbal aspects such as facial expressions, tone, sarcasm, and body language. These elements are integral to human relations, as communication both influences and is influenced by the individuals engaged in it. The way one communicates is shaped by the present social environment and the individuals therein. One key aspect of human relations, as discussed by Duck (2007) in Chapter 5, is their ability to encourage positive behaviours. Relations are built upon communication, which is inherently persuasive. The influence of relations is not only direct, for instance, when a partner asks for a favour, but also indirect, occurring automatically. An example of this is when individuals consider how their actions might be perceived by others, prompting them to reflect: “What would my partner think if they saw me do this?” (Duck, 2007). This form of social persuasion plays a crucial role in encouraging healthy behaviours. Rather than explicitly expressing desires or intentions to others, an indirect approach is often used to subtly encourage others to initiate social interactions. This behaviour is rooted in the fear of rejection, leading individuals to assess the attitudes of potential friends or partners before making direct interactions.

Research shows that shared interests, values, and identities play a crucial role in forming social bonds. According to McMillian and Chavis (1986), a sense of community becomes apparent when individuals feel a shared emotional connection, fulfilment of needs, and a sense of belonging. These elements are more likely to be found when people meet other individuals with similar backgrounds, values or lifestyles (& Chavis, 1986, p. 9) Similarly, Baumeister and Leary (1995) argue that humans have a fundamental psychological need to form and maintain a strong and stable interpersonal relation. They suggest that identification with others based on common characteristics is not only an instigator for social interactions, but also essential for psychological well-being (Baumeister & Leary, 1995, pp. 500–501).

In modern social contexts, which include online communities, people often steer towards groups or individuals who reflect their own interests and identities, where participation is motivated by mutual importance and visibility (Papacharissi, 2011, pp. 39–40). These environments help indirect, low-pressure social engagement, which is particularly beneficial in unfamiliar or transitional life phases such as entering University. This dynamic is

particularly evident in the early stages of friendships, where self-disclosure is employed as a means of deepening connections and fostering mutual affection. This is especially significant in situations where establishing new social connections is essential, such as when relocating to a large city to pursue university studies. We see that communication plays an important role in forming relations, and that relations play a core role in the well-being of students. It enables individuals to become acquainted with one another, develop deeper connections, and navigate the complexities of social interactions, including the assessment of potential friendships (Duck, 2007).

It is evident that human relations play a crucial role in everyday life. However, what are the consequences of lacking social connections on individual well-being? The following section will address this question by exploring mental health and its impact on students and individuals in general.

2.1.2 Community and Mental Health

Mental health is difficult to neatly encapsulate, as it encompasses different stages of existence. A classic example being stress with a focus on the mental burden, for young university students. This is reflected in increased stress due to high performance expectations and an emphasis on grades, with this pressure being internalised, resulting in more people seeking solutions in the psychiatric system (Bylov, n.d).

The Danish health authority connects mental health to general well-being, with their definition of well-being: "the realization of one's potential coping with everyday challenges and stress as well as the ability to participate in communities" (Sundhedsstyrelsen, 2022). There's a strong connection between mental stability and general well-being. People's perception of their struggles in day-to-day life has a significant impact on their ability to participate in social events, increasing the risk of it spiralling out of control.

The Danish health authorities also connect mental health to general well-being using the World Health Organisation's (WHO) definition as a frame to measure mental health using the WHO-5. They split mental health into two dimensions, the *experiential dimension* and the *functional dimension*. The experiential dimension refers to the happiness and the satisfaction of life in general. The functional dimension refers to everyday life such as tasks like cooking, attending school, or participating in social activities. In this context, mental health and well-being are intertwined. Poor mental health will refer to situations where the unhappiness creates a situation that makes it difficult to be social, and in extreme cases complicates small tasks such

as grocery shopping and general self-care. The different factors of mental health are separated into sections, the individual level (feeling of purpose, good physical health and self-efficacy), the social level (family relations, emotional support and community) and the structural level (safe surroundings, financial security and good housing condition) (Sundhedsstyrelsen, 2022). All of these factors can, and most likely will change throughout one's life. All of them come with their own challenges but give a good vantage point to tackle the issue of good well-being (Bruselius-Jensen et al., 2023).

The corresponding connection between well-being and social engagement highlights the risks of self-isolation when individuals struggle to participate in social activities. When students face challenges like academic stress or mental health challenges, their ability to form and maintain relations can suffer, which increases the risk of loneliness. To prevent loneliness, it is essential to give the tools to create social networks and communities that can help young people navigate transitions and integration into new environments, such as universities and new cities (Hartkopf, 2023). In Denmark, loneliness among young adults has been rising, making it a growing concern in the discussion of student mental health. Loneliness is a significant issue among Danish youth, with support from recent studies indicating a substantial increase in its prevalence.

According to a comprehensive survey by the Mary Foundation, more than one in ten Danes experience severe loneliness, with young adults being especially vulnerable. The survey shows that in 2013, the percentage of young adults between 16 and 29 years old who felt severe loneliness was around 6.4%, while recent studies from 2023 show a nearly threefold increase. This figure has risen up to 17% of young adults that have reported feeling severe loneliness (Mary Fonden, n.d). One of the triggers for loneliness amongst young adults is the transition of changing schools or starting university. Over half (55%) of the 16- to 19-year-olds respondents note that their feelings of loneliness are associated with educational transitions (Mary Fonden, n.d).

These findings emphasise the importance of initiatives and tools that help social integration, especially during transition periods. By providing an environment where students can get the tools to form relations, we reduce the possibility of loneliness as well as improve overall well-being.

2.1.3 The Importance of Places and Technology.

Duck (2007) tells an anecdote about their experiences at a Dutch bar, where they were unaware of the customs and offended the guests and the bartender. While the author was simply looking for new experiences in a foreign country, they unknowingly found a social location with an, at the time, incompatible culture. Finding a place with the right social and cultural context is important for one's ability to relate to strangers and potentially build closer relationships (Duck, 2007). The author notes two things:

1. Physical boundaries between places, say a bar and an office per their example, also serve as cultural and social boundaries separating norms of appropriate behaviour and communication.
2. Technologies, which serve as relational tools, break down those boundaries, as communication can happen between people in places where different and contradicting behaviour is appropriate.

Being able to navigate spaces as hubs for distinct cultural and social actions helps one to act appropriately and thus not face rejection. As follows, it would be easier to build relations if you could choose a cultural and social context matching one's sensibilities (Duck, 2007). As technologies increasingly mediate our relations, it becomes important to understand the roles they play in shaping social interaction. We can view technology as being, first and foremost, tools for building and maintaining relations. In this context, we refer specifically to Computer Mediated Communication (CMC). While it has some unique qualities, we do not consider it to be inherently different from Face To Face (F2F) interactions. CMC is culturally influenced in similar ways to F2F interaction. It is evident that CMC users employ both the self-evident and the implied cultural, social, and psychological groups to which they or another person may belong, in order to convey social information or manage impressions, for example by directly referencing their sex, race, nationality or other groupings (Duck, 2007). An important difference between CMC and F2F is the lack of things like body language and tone. One can only communicate with language itself. This barrier must be kept in mind as we look at the possible solutions online technologies and resources that help build connections offer.

We use these CMC and F2F differences to understand the relevance of digital platforms and physical places in fostering student well-being and social engagement. In this reflection, we found it relevant to explore how educational institutions actively support newcomers through

these methods. The following section explores the institutional resources and platforms offered by Aalborg University (AAU) as well as Aarhus University (AU), primarily focusing on how they aim to encourage social interactions, provide support, and facilitate connections amongst their students, specifically those who are new to the city or country.

2.1.4 Social Support and Resources at AAU and AU

As far as initiatives from Danish universities, the availability of resources, varies from school to school. A Google search for “new students university” brings up Aalborg University’s UniStart page as the top result (Aalborg University, n.d) This page provides links to various types of resources related to student life, including information on social activities, recommendations for cafés to visit, upcoming events, and more. Additionally, students can book meetings with a student counsellor for support regarding loneliness or general well-being (Aalborg University, n.d) Although AAU provides a variety of resources, navigating these could be considered somewhat challenging. Students must click through multiple links across several pages to access all available support and social opportunities, which could cause some information to be overlooked. In contrast, AU offers a similar range of resources but presents them in a structured and user-friendly way. Their website consolidates key information on fewer pages, providing a clearer overview of the different topics and options available to students. Topics are displayed in a simple menu, allowing students to navigate through different topics such as ‘student environment’ for social initiatives outside of the classroom and ‘new students’ regarding topics like before starting courses, important days, and events, as well as tips on the transition for studying at AU (AU Studypedia, n.d.).

Both universities also reference external websites to help students expand their social networks. Aalborg University partners with StudieBobler, an extension of Boblberg tailored to students from AAU, AU, and UCN. StudieBobler is described as “a universe where students can connect through shared interests while also getting an overview of activities and student associations on campus (Boblberg, n.d).” Boblberg is a digital platform designed to facilitate social networking and the formation of new friendships. Upon registering, users gain access to a broad range of posts created by others on the platform, allowing them to explore ongoing social opportunities. Notably, user activity remains private unless they choose to create their own post (referred to as a “bobl”) or actively engage with others. This ensures that individuals who are merely curious about the platform’s content can remain anonymous while browsing (Boblberg,

n.d). Unlike traditional social media, Boblberg does not include public engagement features such as likes or comments, nor does it display how many interactions a particular post has received. Instead, communication occurs exclusively through private messaging, where users can initiate conversations and, if mutually agreed upon, arrange to meet in person. This design choice fosters more direct and intentional interactions, accommodating both those seeking in-person socialisation and those who prefer online conversations (Boblberg, n.d)

Upon logging in, users are automatically directed to the homepage, which features a section called “Other Bubbles Near You.” This section displays posts from other users who are looking to connect, often to form new friendships or acquaintances. While the content of these posts varies, many share a common theme of individuals seeking social connections, with a significant number mentioning that they are new students to the city. In addition to personal posts, the platform includes an activity page, which provides an overview of social events and gatherings. This page features an interactive map highlighting areas with the most activities across the country, alongside a structured list detailing each event’s title, location, and date. Users can refine their search by entering specific keywords or selecting dates, allowing them to filter through available activities and find those most relevant to their interests. Similarly, Aarhus University also refers students to Genlyd, an initiative by Aarhus Municipality. Genlyd is an online community platform where people can create and join social initiatives, published on a digital public bulletin board. Unlike StudieBobler, Genlyd is open to all residents, not just students, making it a platform for people of all ages to engage in various communities across Aarhus Municipality (Genlyd, n.d.).

Having available resources displayed visibly could be beneficial for new students seeking acquaintances or knowledge about their new environment. Apart from the external partners and recommendations from the universities, AAU offers a ‘buddy system’ for new international students enrolled in Aalborg University. These existing tools provide an inspiration for systems design and understandings of pros and cons within the existing tools.

2.1.5 Integration as Newcomers

The Buddy system is typically a term, in the context of school, that refers to a contact person or helper who assists with various tasks for a person in new environments. Aalborg University has implemented this system, which consists of a voluntary local student who is paired with two to three international students, to foster a sense of community, introduce the culture, norms,

and humour of the Danish population, and direct and help on practical matters related to their time in Denmark and as a student at AAU (Aalborg University, n.d.).

The main idea behind this concept lies on the integration of international students. However, Danish students who relocate to a new city for their studies often face similar challenges- unfamiliar surroundings, limited social networks and difficulties integrating into student life. Most initiatives in the beginning of the studies in Denmark rely heavily on activities involving alcohol. A study made by Danmarks Evalueringsinstitut states that about 76% of university students consume alcohol during study start, with 60% drinking 6 or more drinks once or twice a week. Non-drinkers thus become a minority, with only 17% of students not drinking at the start of their student time. The reliance on alcohol-centred activities risk excluding non-drinkers from essential social bonding opportunities. Non-drinker report lower levels of social integration, with only 52% feeling welcomed by their peers, compared to 74% of those who drinks the most during the start of the study (Harrild, n.d.). This suggests that social events centred around alcohol may not be inclusive for all students, which may be more difficult for those new to a city yet to form connections. A buddy system for all newcomers - not only international students - could provide inclusive social opportunities, beyond drinking events. By focusing on shared experiences and city exploration, a system could help bridge the gap for students who might otherwise feel isolated.

2.1.6 Key take-aways

There are different important aspects to consider when developing a design for helping student social engagement and well-being. It is important to foster human relations, because feelings of loneliness worsen mental health and inhibit social engagement further. Students new to a city, and those beginning university, are the most vulnerable group of students, and may be further at risk if they do not drink alcohol. Communication is very important for both building and facilitating human relations, and there are strengths and weaknesses to both CMC and F2F communication that mean one cannot replace the other. Physical social spaces mediate social engagement and make it easier to interact over shared interests and experiences. Existing initiatives use these advantages, but the ones offered by AAU and AU can be difficult to discover, and some only target international students. An interactive design with the purpose of helping students form relations could benefit from increasing the discoverability of events and locations relevant to the student.

2.2 Problem Statement

The previous sections gave us insights used as the foundation for the following problem statement:

“How can an interactive system be designed to support students in discovering local events related to their interests?”

With this project, we aimed to answer the problem statement by designing, evaluating and developing an interactive system.

2.3 Methods

In the following section, we describe the methods which we used to explore the problem domain further, emphasising the importance of both Participatory Design as the overarching approach and Design Thinking as the structured, iterative framework that guided our work.

2.3.1 Our HCI design process

This section outlines the methods we used in our *Human-Computer Interaction (HCI)* design process, focusing on user-centred design principles and iterative development.

With a focus on the user in our design process, we looked at the general principles of interaction design, laid out in the book on interaction design by Helen Sharp et al., 2019. These value the user’s input and their knowledge highly. Our goal was to build a product that users can and want to use, thus a product where the implementation supports the user with how they prefer to socially engage.

Throughout this project, we followed a general approach to our design process laid out in Figure 1. The figure presents our process implemented into the Design Thinking Process model (Dam, 2025) including a visualisation of where methods have been applied through the process.

The model consists of 5 phases – *Empathise*, *Define*, *Ideate*, *Prototype*, and *Test*. With these, we split the model into two groups: *Problem Domain*, and *Application Domain*. These 5 phases are sorted into the problem domain and the application domain respectively. The problem domain consists of the first two phases and focuses on understanding and defining user needs and problems within the project scope. It is defined as “The parts of a context that is administered, monitored or controlled by a system” (Mathiassen et al., 2018, p.6), where *context* refers to a real-world environment and *system* refers to a group of people coordinating.

The last three phases focus on the application domain, which means that they are focused on the system made to administer, monitor or control the context (Mathiassen et al., 2018).

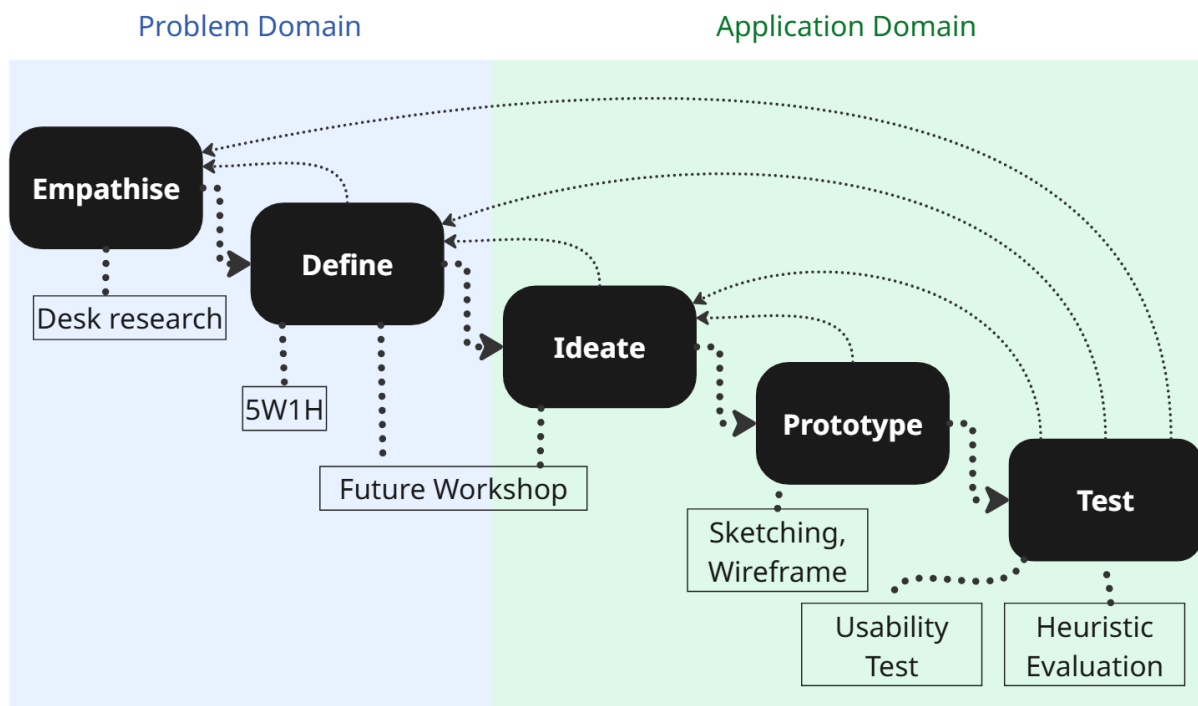


Figure 1 Design Thinking Process model of our iterative design

Figure 1 shows the design process of this project. Our design was an iterative process where we often went back and re-evaluated previous steps. The smaller arrows in Figure 1 highlight where we often looked back to re-evaluate. Using an iterative process was essential to the relevance and effectiveness of our design, as we were able to apply the newly found knowledge and experience to continuously improve each step of the process. By utilising this flexibility, we could make a solution that was innovative and evolved in response to user input, which was essential when identifying system requirements.

At this stage of our project, our focus remained mainly on exploring and understanding the problem domain rather than developing concrete solutions. We drew on our own experiences as students throughout the design process and used a participatory design approach from the Design- to the Prototype phase using a future workshop and a usability test. This allowed us to involve students, not just as users, but as collaborators. We followed this approach with the intention of our design process being both structured and flexible, and that the resulting system would be meaningfully informed by students' real-world experiences. To explore and understand the problem of newcomers struggling to establish meaningful human relationships

through events in Aalborg, we combined a variety of methods including desk research, interviews, workshops, brainstorming, and prototyping.

After gathering initial insights, we transitioned into the Define phase. To structure the knowledge from our research and interviews, we applied the 5W1H method, which helped break down the problem by asking: Who is affected? What are the challenges? Where do these challenges occur? When are students most vulnerable? Why are current solutions failing? And how can we potentially intervene? (Venkatakrishnan, 2021). This analysis enabled us to clarify our problem formulation and narrow the design scope.

Following this, we transitioned into the Ideate phase, where we facilitated a workshop involving participants from the target group. Through open dialogue about their experiences and ideas for possible solutions, we gained further insights into the problem domain and were able to initiate the ideation of potential solutions. Building on the participants' input, specifically their expressed needs and expectations, early system requirements started to appear, shaping the direction of our brainstorming activities, and later informing prototype development.

2.3.2 Participatory Design Approach

Participation is, at its core, a way for people to influence digital technologies that changes their everyday lives and work practices (Bødker et al., 2022). In participatory design people will actively be a part of the process of imagining their own future work practices, communities, and everyday life through the design of digital technologies (Bødker et al., 2022). Our aim is to engage people to realise or support their imagined future technologies, giving the participants a choice and reflect on their own relationship with every day digital technology, such as smartphones, tv and computers. Therefore, participatory design is directed toward the tangible design outcome, in relation to future applications, system or hardware, where having a critical and inquisitive approach to digital technology is important among its participants. We focus on Bødker's definition of participatory design, which is: "Participatory design in our definition includes activities where users, designers, and researchers collaborate toward shared goals" (Bødker et al., 2022).

We chose participatory design as a design approach based on the paper's main focus, which is students' everyday life and communities. Our focus was to understand and apply students' personal experiences and choices when they moved to Aalborg and started their educations at

AAU. Specifically, we explored how they built new social connections in unfamiliar environments. Hence, we wanted them to participate in designing a system that would support new students' social lives and help them learn more about their new environment.

Mutual learning is important to our understanding of student's current practices when choosing events and building on social connections. To explore students' current practices and experiences, we conducted a group conversation in the form of an open interview. This initiated a discussion on how the students identified events of interest, aiming to establish a sense of community as new students (Bødker et al., 2022).

The workshop method was a way to ensure the students' participation in showing us established systems they used for building human relationships and how effective their current practices were. We used the requirements acquired from the workshop and our own experience as a base for sketching and prototyping. We applied several participatory methods that fostered ongoing collaboration and learning throughout the design process.

These methods facilitated a broader exploration of the design process while deepening our understanding as well as applying the users' insights. The following sections of the report will explain, in a chronological order, the different methods of participatory design that were employed, throughout the different stages of the process. Among these, brainstorming activities proved especially valuable in encouraging collaboration and creative thinking during the initial phases of our project.

2.3.3 Brainstorm

As part of our problem domain development, we used several brainstorming activities to explore and generate ideas. These sessions played a central role in shaping our understanding of the problem domain and helped us move from broad themes to focused problem areas.

Following the workshop and interviews, we revisited brainstorming to explore how the new uncovered information and insights could inspire more of a targeted design idea. This post-workshop brainstorming allowed us to further connect user input with a possible direction for a solution. In this way, we found ourselves still operating between the Define and Ideate phases, using brainstorming both to clarify problem areas and start early concept development.

We used brainstorming as an informal verbal technique where all group members were encouraged to share ideas freely, without immediate judgment or evaluation. This approach supported a collaborative and open environment, where we were able to build on each other's

contributions and allow creative ideas to organically emerge (*Interaction Design Foundation*, 2016). We applied the initial brainstorming throughout the first stage Empathise. By having no set theme to go from, the brainstorm started from absolute zero with a whiteboard and a marker. Topics ranged from physical artefacts to non-tangible programs, with subjects ranging from computer games to eldercare. By narrowing down the topics, we discovered a common denominator: Something that helps somebody. This process can be seen in Figure 2 below all the different topics we discussed as interesting. The topics were discussed in Danish as written on the whiteboard.

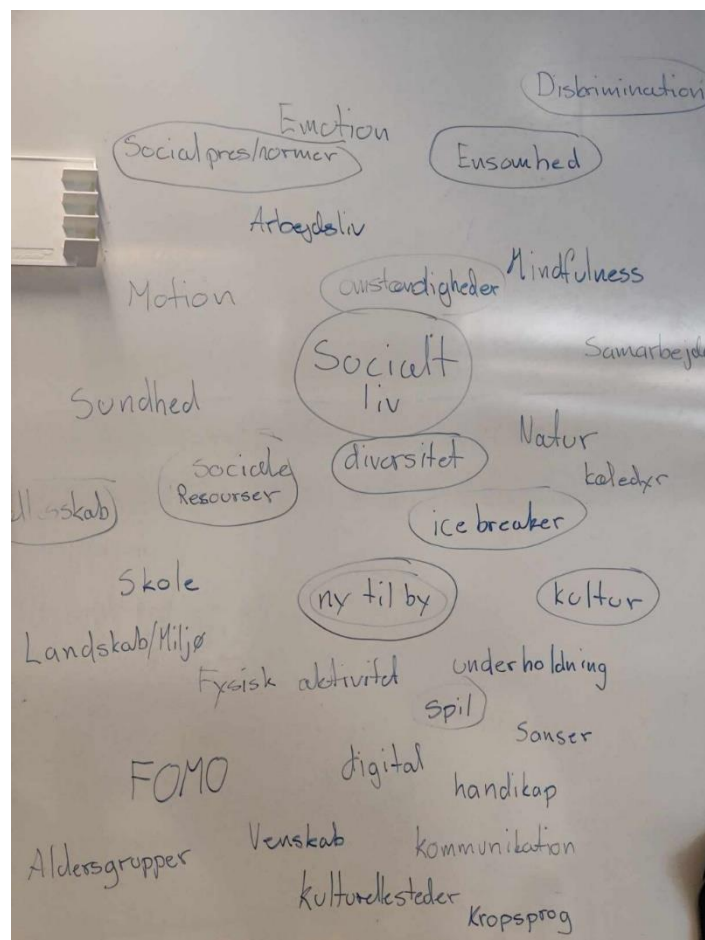


Figure 2 Brainstorm

We discussed our own experiences of needing help, what we needed help with, from whom, in which scenarios, and what we found interesting about it all. We were specifically interested in university students feeling lonely and the reasons why. We considered how loneliness affected new students who relocated from other locations and how a lack of a social network contributed to that experience.

We decided on the broad topics of loneliness, cultural barriers, and norms. We used these topics as a starting point to do the 5W1H, aiming for a better understanding of the problem domain.

2.3.4 The 5W1H

To help organise and evaluate the ideas generated during our Define phase, we employed the 5W1H framework. It helped us break down the complexity of the problem domain by prompting us to consider users, actions, needs, locations, contexts, and processes related to the problem domain. The 5W1H method is a practical tool used by architects and designers to address design challenges in the form of a framework, guiding designers' way of thinking throughout the design process (Venkatakrishnan, 2021). By looking over six key angles: *Who*, *What*, *Where*, *When*, *Why*, and *How*, also illustrated in Figure 3 below, we could develop a comprehensive understanding of the problem. This structure helped us break down the problem, with the intention of the final solution being both functional and user centred.

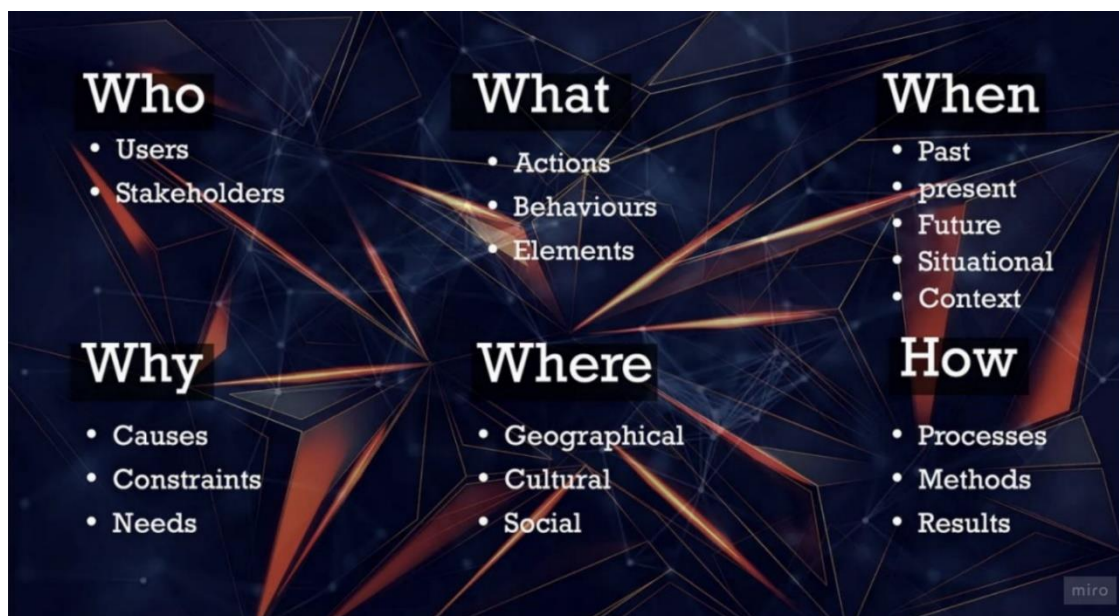


Figure 3 5W1H method

Good design is about understanding the user, which goes beyond technical knowledge or observation. By asking questions that explore the needs, emotions, and experiences of users, we could turn data into meaningful, impactful solutions (Helen Sharp et al., 2019). This approach helped us to not only meet the functional requirements but also connect with the people who would use what was being made.

The questions can be applied at various stages of the design process, from understanding the brief and defining user needs to prototyping and refining the final design, ensuring that solutions are both effective and user-friendly (Venkatakrishnan, 2021).

Each topic was explored through Post-it notes on a digital whiteboard, where we grouped our insights such as “students struggling to build new relationships,” “lack of emotional support,” and “meeting people through social events.” (See Figure 4). These insights were then revised and evaluated with dot voting. Dot voting is used to identify the most relevant themes in a democratic manner, where each participant was given virtual dots to place on subjects, they found most important. (Dot Voting, 2019) This allowed us to collectively prioritise areas of interest and clearly see where the group’s focus naturally converged. For example, topics like “loneliness”, “lack of social networks”, “cultural barriers”, and “need for practical support” received the most votes and became central to framing our problem. This process can be seen in Figure 4 below.

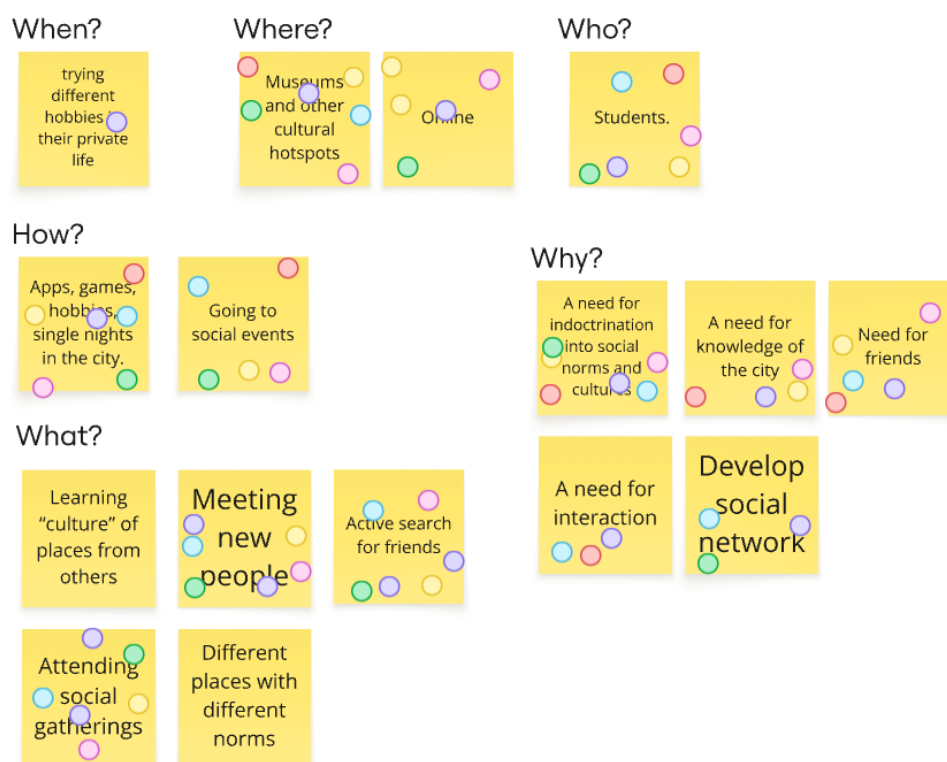


Figure 4: Problem areas using 5W1H and dot voting

These results became the foundation for the next step, where we conducted an open group interview in combination with a workshop.

2.3.5 The Open Group Interview

To explore the initial stage of the problem domain and to guide the early stages of the Ideate phase, we conducted a group interview using an unstructured, qualitative approach. The interview consisted of several key questions defining the areas we wished to explore, which also allowed both interviewer and interviewee to diverge. This allowed the possibility of pursuing ideas or responses in detail. The interview was meant to explore other directions of the concept, while also helping uncover any gaps in our understanding of the problem domain. Addressing these perspectives when they emerged helped us ensure that we were focusing on the right challenges.

The interview was conducted as a focus group, which provided information on collective views. The composition of the group of participants in a focus group play a crucial role in allowing discussion (Kvale & Brinkmann, 2022, pp. 205–206). Therefore, the participants for the interview were selected based on having a mix of international and Danish students. The main common denominator was that they were all students at AAU. There were six participants in the group to facilitate open discussions and create a space that allowed for free-flowing conversation. All participants were informed of the intention of the interview and gave consent to use their data anonymously in the results reported in Section 1.5.

The interview questions were created to gather knowledge regarding their experiences of being newcomers and how they discovered and attended social events. The interview consisted of two group members asking the questions and two group members assisting as note-takers. The data from the interview was combined with insights from a workshop to identify key patterns relevant to the ideation process.

2.3.6 Workshop

Following the contextual framing established through the group interview, at the beginning of the workshop, we proceeded into a *Future Workshop*. The Future Workshop is a foundational method in participatory design, particularly useful for data collection involving user engagement and idea generation (*Future Workshop – Participedia*, n.d). While we initially planned to follow a traditional Future Workshop structure, the actual workshop process evolved into a more flexible and conversational format. Although we drew on the key elements of the method, critique, fantasy, and implementation, the session did not follow the phases in a strictly

linear manner. Instead, the structure of the workshop adapted to participants' input and natural flow of dialogue. This shift is described in more detail later in this section.

Our participant group consisted of university students from various academic levels, brought together to engage in a discussion regarding how they discovered and participated in events. The Future Workshop consists of three distinct phases: the Critique Phase, the Fantasy Phase, and the Implementation Phase (*Future Workshop – Participedia*, n.d).

In The Critique phase, our aim was to identify and articulate specific problems and questions related to our central research problem, with a focus on understanding the issue from the users' perspectives. This was facilitated through an interview, where participants were encouraged to express their critiques surrounding existing systems. These critiques were documented using notetaking and writing on Post-it notes. To establish a shared context, participants were asked to reflect on and evaluate existing systems for event discovery and social interaction platforms, highlighting what they deemed to be the strengths and weaknesses.

During the Fantasy phase, participants were encouraged to openly ideate how, why, and what new digital solutions could better support the formation of human relationships and help discover social events. Their ideas were captured and visually organised on a shared board, supporting the identification of common themes and directions for the design process.

Finally, in the Implementation phase, the focus shifted to concretising the most promising concepts or ideas. Participants were encouraged to define potential system requirements and identify feasible elements that could form a foundation for the development of a new system (*Future Workshop – Participedia*, n.d).

Although we planned the session around the structure of a traditional future workshop, with critique, fantasy, and implementation phases, the actual process evolved into a more fluid and dynamic dialogue. Rather than following the method step by step, the discussion unfolded more organically, shaped by the participants' input and the informal setting we created to support open conversation. Instead of separated phases, some of the themes from the critique and fantasy phases blended during the dialogue. This shift allowed us to follow up on ideas as they emerged, without interrupting the flow of the conversation.

Although this approach did not follow the guidelines for the Future Workshop format, the session still reflected its core goals: user participation, mutual learning, and collective idea generation. Written notes and tools like Post-it were used to support the process (see Figure 5)

by visualising and displaying ideas and topics during the workshop. In this way, the workshop became a mix of future workshop elements and informal co-creation, tailored to fit the process. The insights from the workshop, along with the data from the group interview, helped us identify key themes and define initial system requirements. This gave us a clearer understanding of the problem domain and guided the direction of the design process.

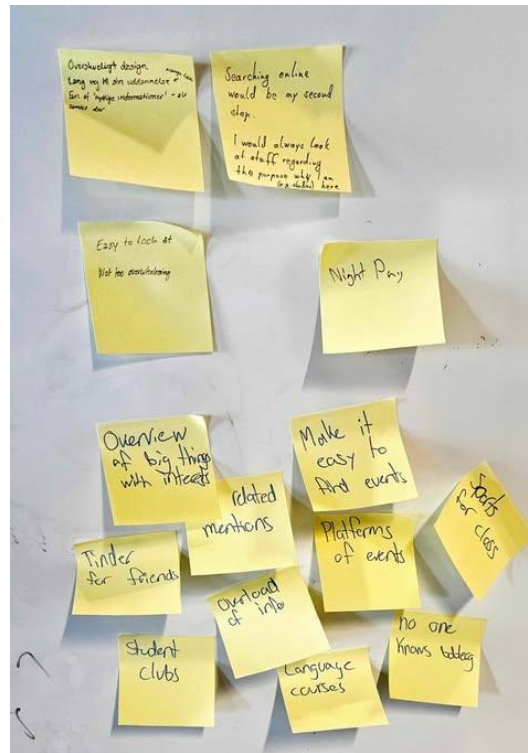


Figure 5 Post-it notes as a result of our workshop

After the interview/workshop was conducted, we could move from defining the problem domain to ideation in the application domain. Students were involved from the early stages and in through to testing. The resulting concept development was derived from the data collected in the interview and workshop.

3 Application Domain

3.1 Concept Development

The following section describes the development of the final concept by highlighting the transition from initial insights and data analysis to ideation, design choices, and the prototyping of our solution. By outlining user-centred and participatory design methods, this process aimed to transform raw collected data from qualitative insights gathered in our interviews and workshop, into system requirements and a coherent digital interface. By applying thematic analysis to collected qualitative data, prioritising functionalities using the *MoSCoW* framework, and interpreting recurring patterns, we managed to shape a concept that addressed the identified needs of users. The section delves into how these insights helped guide us in the construction of the character-based personas, which served as the navigational guides within the app, and how these personas were further developed and refined through iterative prototyping and user feedback. Our concept development process began with data analysis, which served as the foundation for identifying user needs and defining key system requirements.

3.2 Data Analysis

The processes of Coding of qualitative data in which information collected from interviews or observations is segmented into meaningful parts and assigned descriptive labels. This process involves identifying and organising themes within the dataset by extracting key statements, applying generalisation, and interpreting the underlying meaning of the data. This is also known as thematic analysis (Aarhus University, n.d.) . The data can be accessed in Appendix 1.

The conducted workshop and interview, during which participants shared various perspectives on their use of systems to discover events in Aalborg. They provided critical feedback on existing systems, discussed platforms they currently use or previously used upon moving to the city. They offered suggestions regarding the features and functionalities they believe a system should have to be more effective (Aarhus University, n.d.).

Participant statements were categorised into thematic codes such as “To meet around something”, “Facebook/communication” and “Digitalisation”. The themes encompass a collection of related statements that reflect a shared idea or concern. The thematic coding was carried out to simplify and structure the data, thereby making it more manageable for us to extract system requirements (Aarhus University, n.d.). As seen in the following table:

Table 1 Themes

Theme	Statements based on interview and Post-it notes
Meeting over some activity	- Aalborg try-outs was an example of a way to try events around the city, they learned about it from an international student's fair. - Alcohol can be a tool to creating communities, in the student house or quiz nights they are easy to participate in. They use those places as a meeting point. - Quiz nights student house, Jomfru Ane Gade student household events which is a bit expensive. - Not the AAU sites for events, me and Aalborg comes up and a lot of articles from AAU. - Texting someone that already studied here, got to know about the student house. - Creating a social network with his roomies gave him a tool for starting social relations. - Moved up here with a friend of his as the only one he knew, went out started drinking and just met people that way. People that are drunk is easier to talk to, roommates' and collectives it helps living in centrum which helps the process. - Matching interests with people with the same interests. - Trying out different sports that could be implemented more.
Facebook/communication	- About how difficult it can be to find the event you're looking for, the 'Relevant for you this week' section does not show either the ones you are interested in or the ones you are searching for. You have to find the right tab to locate the one you have marked as interested. Otherwise, you have to search for the specific venues that are hosting events. - Facebook is not big in all countries. - People message on Facebook so they had to get it again. - Event flyers, link to Facebook through messages. - Facebook, which pages are interesting for events, and which hold what and what is exciting to you and so on. - A lot of word by mouth.
University	- Some masters are forgotten. - The front page of AAU is simple and seems like there are not a lot, but you can click around a lot. - University has a lot of different sites that comes forth. - There is a survival guide for international student on AAU websites. - AAU applications are not necessary for everyone, because you can get all the relevant information from your tutors.

A notable recurring pattern theme in our findings is the reliance on informal channels, particularly word-of-mouth and peer tutors. More experienced peers are typically responsible for ensuring that newcomers are informed about relevant events, communities, and aspects of the city. This insight is exemplified in participant statements such as:

- “A lot of word of mouth” (Appendix 1).
- “People message on Facebook so they had to get it (redownload) again” (Appendix 1).
- “Moved up here with a friend of his as the only one he knew, went out, started drinking, and just met people that way. Drunk people are easier to talk to, roommates and collectives help living in the city centre facilitate the process” (Appendix 1).

These statements collectively highlight that individuals new to Aalborg often rely on more experienced peers to navigate the city and its social offerings, rather than exploring on their own. Exploring with another newcomer is also preferred to exploring events and the city on their own. This observation is significant, as it suggests that facilitating social integration for newcomers may depend on providing them with a relatable guide that could introduce them to relevant events and communities based on their interests.

The data analysis clarified recurring patterns from which we were able to define a set of system requirements used as a guide in the design process. To structure and prioritise these requirements, we applied the MoSCoW method, which allowed us to categorise features into four levels of importance: must-have, should-have, could-have, and want-to-have (*The Interaction Design Foundation*, 2015). This prioritisation helped us focus on the most essential elements while still allowing room for creative interpretation in the design.

3.2.1 MoSCoW Requirements

The MoSCoW requirements shaped the beginning of defining the application domain. We began to build an understanding for how we might best administer and organise the data in our application domain (*The Interaction Design Foundation*, 2015).

The primary justification for each requirement is based on notes taken during the interview/workshop session, and user-generated content such as Post-it notes. While existing knowledge from desk research has influenced our interpretation of the data, it has not been explicitly used as a source of argumentation for the requirements. These are prioritised

requirements where we follow MoSCoW, which is an abbreviation for must have, should have, could have and want to have (*The Interaction Design Foundation*, 2015).

The resulting system requirements are presented in the table below:

Req no.	Functional Req. Name	Description	Argument from users	MoSCoW
1.	Guided City Exploration for Users.	The system must allow users to follow a character-based or point-based guide, enabling them to explore and become familiar with the city without getting lost.	“Moved up here with a friend of his as the only one he knew, went out started drinking and just met people that way. People that are drunk is easier to talk to, roommates and collectives it helps living in centrum which helps the process.”	Must
2.	Matching users with events through interests.	The system must allow users to view and select events that match their specified interests.	“Make it easy to find event.” “Overview of big things with interests.” “About how difficult it can be to find the event you're looking for the 'Relevant for you this week' section on FB doesn't show either the ones you're interested in or the ones you're searching for.” “Matching interests with people with the same interests.”	Must
3.	User login	The system must allow users to login and choose a character based on their own interests	See argument from requirement number one.	Must
4.	Choosing events	The system should allow students to choose events and view details.	“Facebook, which pages are interesting for events, and which hold what and what is exciting to you and so on.”	Should

5.	Saving events	The system should allow students to save events through their profiles.	“About how difficult it can be to find the event you're looking for the 'Relevant for you this week' section doesn't show either the ones you're interested in or the ones you're searching for. You have to find the right tab to locate the one you've marked as interested. Otherwise, you have to search for the specific venues that are hosting events.”	Should
6.	Deleting events	We want the system to allow students to remove events from the calendar in the application.		Want
7.	Adding events to own calendar	We want the system to allow students to add their events to their personal calendar.		Want
Req no.	Non-functional Req. Name	Description	Argument from users.	MoSCoW
8.	Overload of info	The system must be easy to navigate and not give too much info on one page	“Overload of info.” “Clear design on the AAU website, long way to your education (many clicks), fan of "useful information", everything is gathered there.”	Must
9.	City exploration through activities.	The system must enable users to explore the city by offering a variety of activities designed to introduce them to the local culture and environment.	“Trying out different sports that could be implemented more.” “A lot of different events, bowling as an event, the buddy program they function as guides. The Aalborg try-outs socialized a lot.” “It is efficient that the city is smaller it is easy to go around, and digitalization works to help you come around and experience things.”	Must

Table 2 MoSCoW requirements

The requirements above are abstract which leads to an open interpretation and ideation for the design, while still being based on statements that led us to some definitive aspects that the system should have. Requirements as number 6 and 7 is quality of life requirements that does not necessarily have an argument in the data from the workshop but is considered as a “should have” feature that would make the system more user-friendly. Otherwise, the requirements are being kept broad so we as designers can interpret and make a design that makes sense with our understanding of the problem domain and the data. Therefore we, decided on creating personas.

3.2.2 Personas and Characters

We conducted a brainstorming session after getting feedback from lecturers and fellow students to accommodate the requirements derived from workshop data and interview insights. The feedback consisted of a need for something more “fun” and “interactive”. Continuing the ideation phase, we then developed the concept of using personas designed to assist new students in identifying events aligned with their interests. We decided to include personas to create a distinct user experience, aiming to avoid our concept resembling event-focused sites, platforms such as Facebook, which were mentioned by participants during the workshop (Appendix 1). Known limitations of participatory design, especially workshops and interviews, involve participants being unaware of the full range of design possibilities. With that in mind, we allowed ourselves some creative freedom, using the data and problem domain as a starting point for the design, rather than a constraint. The personas helped us capture key themes from the workshop, such as “word of mouth” and “getting to know the city through someone.” By developing fictional characters that users could “match” with and “follow”, we aimed to encourage city exploration and community discovery through events tied to different kinds of interests.

We employed *ad hoc personas*, constructed based on intuition and stereotypes based on our own experiences as individuals in the university community and as residents of Aalborg. These personas enabled us, as the designers, to maintain an empathetic perspective throughout the process. While personas are often used as a method to gain insight into the problem domain, our approach focused on creating characters derived from recognisable stereotypes and personal experience. This allowed us to address the identified non-functional requirement of recreating a “buddy” experience, like ones available through AAU. Our use of personas aimed to align with Benyon’s perspective on how personas can be created to stimulate design thinking and maintain an empathetic focus during system development (Benyon, 2014, pp. 55–62).

To make the personas engaging to users, we translated them into fictional characters. Each character had a unique name, backstory, and a category of event and venue preferences. This embodiment helped us to bring the personas to life, aiming to make them relatable to users. It also supported the themes of “word of mouth” and “getting to know the city through someone,” which some participants from the workshop highlighted as meaningful based on their own experiences as newcomers. We grouped the personas into five broad lifestyle categories: social, artsy, sporty, geeky, and musical. Each category was represented with a corresponding character, which was how the five personas were displayed in the prototype. The characters were the focus of the prototype, aiming to help users discover events that aligned with their interests.

By associating each persona with specific types of events and venues, based on our experiences and recognisable stereotypes, we aimed to simplify navigation and promote exploration. While each persona is rooted in a distinct area of interest, we wanted to allow users to switch between the different categories, as the personas were very specific in the kind of events they contained. We also recognised that real users' interests could overlap with several personas, as different events naturally appeal to multiple types of people. This insight made us consider how we could enable users to move freely between the different personas and explore other types of events.

The five personas and their corresponding characters can be seen in the table below (Table 3), along with the related locations and events.

Tipsy: Social	Sketchy: Artsy	Rally: Sporty	Glitch: Geeky	Bongo: Musical
Aalborg Regatta	Trekanten	Portland Park	Spilcaféen	Lakridspizza
Student House	Kunsten	Racing Arena Aalborg	Dice ‘n drinks	Læsehesten
GAME Streetmekka	GAME Streetmekka	GAME Streetmekka	LAN parties	GAME Streetmekka
Rekom Pubcrawls	Karolinelund	Nordkraft	CREATE	Tusindfryd
Public screenings of sport events	DuErSmyk	DM Week	CEAGAR Game Lab	Det Hem’lige Teater

Fredagsbar: Aalbar etc.	ArtByMe	Vestre Fjordpark	AKKC Student House	Rocken El mundo
Salling Rooftop	Street Art Aalborg	GIGANTIUM	Det Hem'lige Teater	Music quiz
Vestre Fjordpark	HUSET i Hasserisgade	Padel lounge Miniature golf	Aalborg Tryout	Musikkens Hus
Østre Anlæg	Art Galleries	Klatreparken Aalborg	Vikingsmuseum Lindholm Høje	Aalborg Regatta
Jomfru Ane Park DJ	KREA-Messe	Disc Golf	Library	Skovrock Band Battle
Fredagsfest i Karolinelund	The Cinema	Skatepark Hesteskoen	Buens Bogcafe	Aalborg Karneval
Skovrock AKKC	Buens Bogcafe	Aalborg Tryouts Unirun	Aalborg Forsvars- og Garnisonsmuseum	Karaoke Live Music
Aalborg Pride	AKKC	Aalborg Cable Park	Gråbrødre Kloster	Studiestartsfesten
		Skovdalen Atletikstadion	Læsehesten	AKKC
			The Cinema	

Table 3 Personas and corresponding events and locations

The table lists event and place names in English, while the official venue names remain in Danish. Based on the identified differences between the five personas, we assigned each a specific category (social, artsy, sporty, geeky, and musical) and a corresponding character: *Tipsy*, *Sketchy*, *Rally*, *Glitch*, and *Bongo*.

Tipsy represents the "social" persona, designed for individuals who enjoy social activities such as grabbing a coffee at a new cafe, attending quiz nights, or having a beer at a random bar. *Sketchy* represents the "artsy" persona, who likes creative and cultural events such as pottery-making, painting, and visiting art museums or venues that host artistic experiences. *Rally* represents the "active" persona, associated with outdoor and physical activities, such as football and disc golf. *Glitch* represents those with a "geeky" persona, encompassing interests such as historical events, gaming events, and intricate board games. Finally, *Bongo* represents the "musical" persona, connected to events such as concerts, band battles, and musicals.

By using these ad hoc personas as the foundation for the prototype’s characters, we aimed to represent a diverse range of real-world users with varying lifestyles and interests. Using these personas as the foundation for the characters helped refine the design requirements for a prototype. For the sake of maintaining clarity and consistency, we will refer to the characters throughout the project, rather than personas. While the characters are based on ad hoc personas developed during the design phase, users engage directly with the characters, and not with the underlying persona models. With the characters categorised and system requirements defined; we were ready to begin the next design phase.

3.2.3 First Iteration of the Prototype

Having established the characters, we shifted our focus toward translating them to visual elements (*The Interaction Design Foundation*, n.d.). This marked the transition from ideation to prototyping within the design thinking process. Users do not always know what they want or need in a product before they get to use it, which is why prototyping was an important tool for us as designers. (Helen Sharp et al., 2019, p. 422). We began the iterative process of sketching early design concepts, aiming to create a concrete foundation for users, something they could engage with and provide feedback on (Helen Sharp et al., 2019, p. 422). We began the process by using generative AI as a brainstorming tool to explore visual ideas for the characters (*Nielsen Norman Group*, n.d.-b) This allowed us to quickly experiment with and change different elements of the designs, helping us evaluate what visual direction to pursue.



Figure 6 Topsy’s different design iterations

Having the five personas in place, we generated prompts asking for a visual description if based solely on their stereotypical traits. We generated several visual iterations using those prompts, such as our chosen prompt for the social persona: ”Generate a social 2D cartoon character

named Tippy, animated with clear lines using the base colour #F630AB (vibrant pink). Her personality is flirty and bubbly, wearing fun summer clothes, holding a drink with a small umbrella. Rounded features, soft curves, cheeky smile, slight tan lines.”.

The figure above pictures some of the different iterations of the social character Tippy. At first, the character resembled a young, animated woman, evolving to be a 2D cartoon, and lastly as an animated parrot. These helped us translate the abstract persona traits into visual representations. Each character followed the same iterative process, resulting in a set of five distinct and visually engaging characters, as displayed below (See Figure 7):

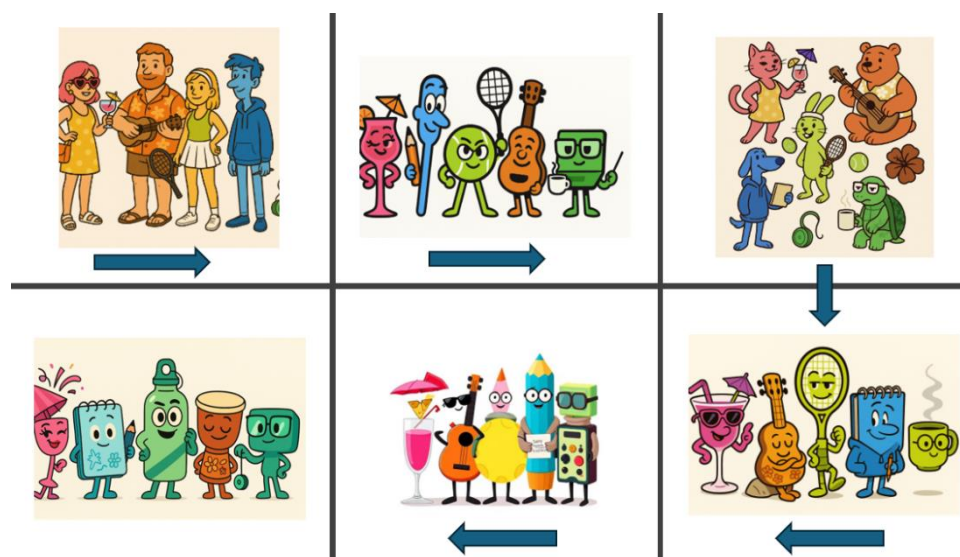


Figure 7 Design iterations of the characters

The initial design of the characters featured a cartoonish but human-like style, looking to emphasise the connection between characters and users. However, some of the designs included excessive details, which risked being visually overwhelming. This prompted a design process involving multiple iterations exploring different styles. We eventually settled on a balanced approach, where the characters resembled objects related to their category of activities, with the inclusion of human features like faces, arms, and legs. With this design, we aimed to make the characters distinct enough to easily differentiate between them, while still being approachable and recognisable to users. During the process, we discussed each iteration and adjusted the prompts accordingly. The last iteration of design suggestions of the two characters Tippy (far left in Figure 7) and Glitch (far right in Figure 7), did not initially align with our developed vision, as we felt that the generator failed to capture their intended personalities.

As a result, we further refined their appearances through paper sketches, such as the one presented below in Figure 8. The sketches helped visualise the complete picture of the characters, helping ensure that they were consistent in size and style. By sharing the sketches during group discussions, we incorporated our different perspectives and reflections on the designs, which contributed to shaping the final visual direction (*The Interaction Design Foundation*, n.d.). During these discussions, we also discussed the design of other interfaces in the first prototype. These discussions were only verbal and are not shown in this report.



Figure 8 Paper sketch of characters

The paper sketch provided an outline for the final touches of the character designs, but the characters still lacked colours. Colours impact on users is complex and often subjective, which led us to base our choices on an interpreted colour wheel that associates different colours with specific expressions and moods. (*Toptal Design Blog*, n.d.) The colour of each character was hence chosen to represent the characteristic traits of the characters, just like how the objects they resembled reflected their category of activities.

Following the transitions from black-and-grey pencil sketches to digital designs, colour became a key element in creating a more engaging user experience. The colour each character was given was vibrant and recognisable, which not only added to their personality but also aimed to enhance the user experience through aesthetics.

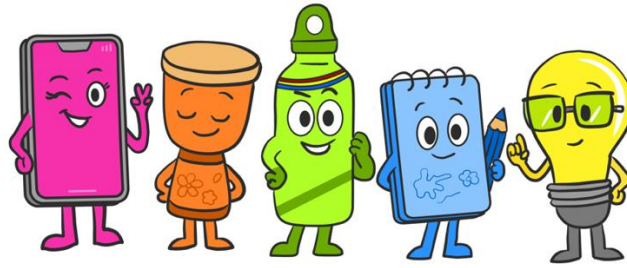


Figure 9 Digital coloured sketch of characters

With the character designs finalised, the next step involved translating the system requirements from the workshop into different pages and interfaces of a second prototype.

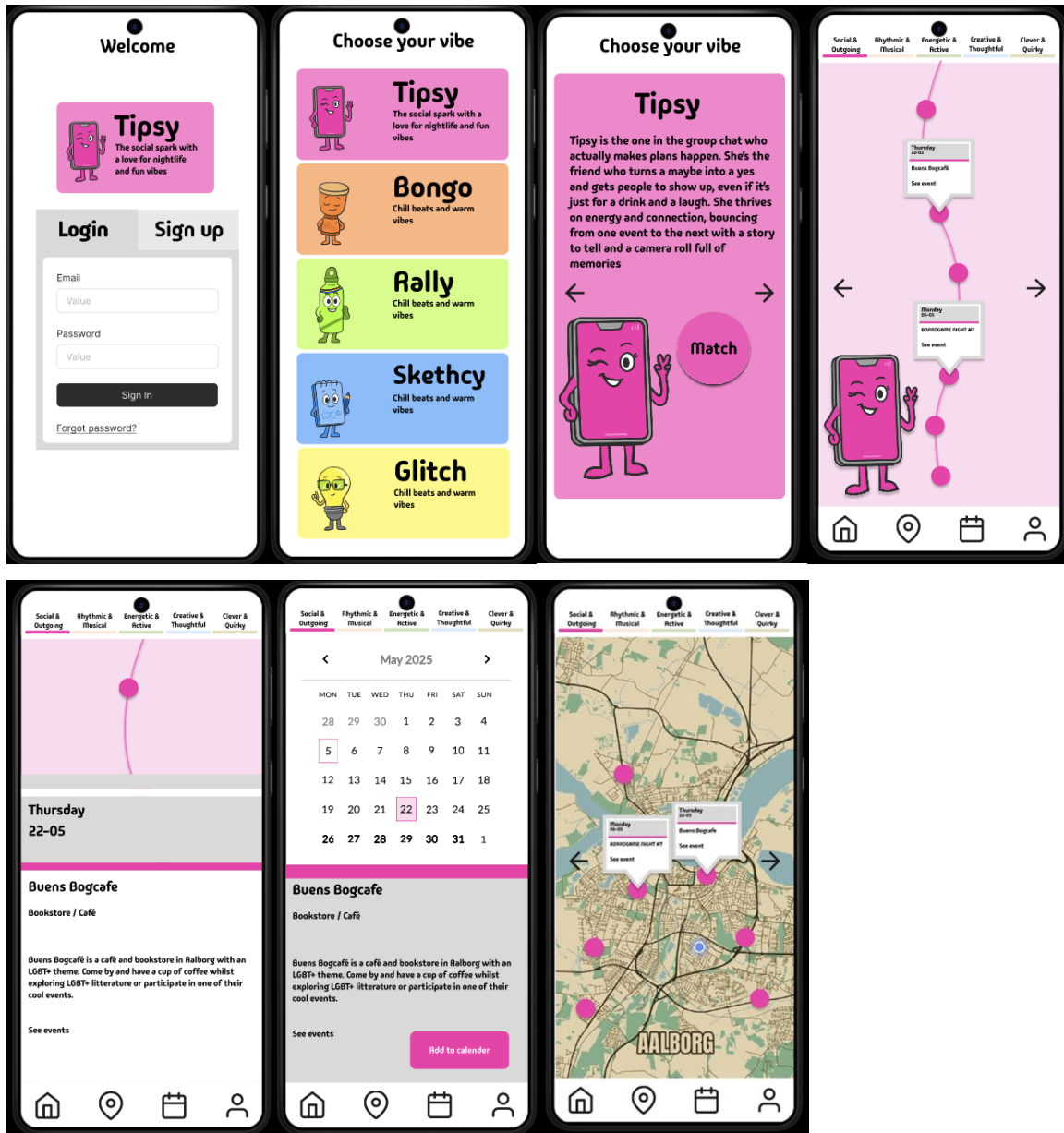


Figure 10 First iteration of high-fidelity prototype

Drawing inspiration from the visual layout of Duolingo and Candy Crush, we combined the ideas of a path and a character guiding the way. A ‘Must’ function from our system requirements detailed the need for a calendar, as well as some sort of map displaying the locations of events. We applied this to create a layout on the user interface that we deemed to fit established industry standards. Here, a user is first met with the login and sign-up interface (Req 3. User login), and after logging in, the users are introduced to the characters for the first time, with an overview screen titled *Choose your vibe*, with all five characters are listed. Each features a distinctive visual identity and a short description of who the characters are and what things they like to do, giving the users a chance to choose the “vibe” they resonate with most. After choosing a character from the list, users are presented with a more detailed description

of the selected character, allowing the user to reflect on how well the description aligns with their own interests. At this point the users are prompted to “match” with the character, which would bring the user to that character’s timeline of events, represented as a path (Req. 2, Matching users with events through interests). We aimed to establish users with a connection to their selected characters, intending to lay a foundation for a more personal experience.

Each page in the second prototype was designed with its character’s assigned colour, aiming to support easy recognition and comparison. Furthermore, the character selection is not fixed, the users are able to revisit the profile section at any time using the app and select a different “vibe” if their initial choice did not align with their preferences. When a user selects a specific character, the interface adapts. The events, suggestions, and other relevant content is colour coded to the selected character. From this point, the user could navigate the interface by using a navigation menu in the bottom of the interface with four main functions: *Home*, *Map*, *Calendar*, and *My Profile*. (Req. 8, Overload of info). The *Home* button links back to the list of characters, while the *Map* button displays an interface that highlights events, colour-coded and tailored to the selected character. The *Calendar* gives users the possibility to save events, integrating them into their personalised schedule (Req.5, Saving events). Clicking on an event also reveals further details about the event and/or activity (Req.4, Choosing Events)

3.2.4 Second Iteration of the Prototype

The first iteration of our high-fidelity prototype gave us a vantagepoint to establish foundations for the application, in the next section we aim to the heuristic principles as a framework for our usability test, after which we gather our insights and analyse them using a deductive thematic analysis. This will give us the opportunity to make further adjustments to our prototype by including feedback in our design.

3.2.4.1 Heuristic Principles as a Tool

After creating the first draft of the prototype, we recognised the need for a more structured and documented approach to justifying the design decisions. The initial sketches of the prototype were developed mainly by members of the group with an academic background in Art and Technology, relying heavily on intuition and past project experience. As the project progressed, intensive group evaluations combined with tools and methods from lectures on Designing User Interfaces were prioritised in the design, allowed us to take a more methodical approach. These tools influenced the development of the prototype and became an integral part of how we refined and evaluated our design choices, with one of those being Nielsen’s Heuristic Principles

(Nielsen Norman Group, n.d.-a). For the User Interface Design, we aimed to create a simple yet intuitive user experience, using Nielsen's ten usability heuristics as inspiration for both our usability test and the improvements made to the prototype. These principles helped lay the foundation for the visual aesthetics, core functions, and overall navigation of the prototype. While we acknowledge that the heuristics are traditionally used by experts to evaluate interfaces, we also applied them as inspiration to formulate questions for a usability test. The ten principles and the questions they inspired can be seen in Table 4 below:

Heuristic principles		Inspired questions
1.	Visibility of system status: The design should keep the user informed.	Did you feel confident finding your way around the application without instructions? How did you know which buttons to press? Did the application behave as you expected when you pressed a button? What was your experience like when exploring the characters / calendar / timeline / profile?
2.	Match between system and real world: use concepts that is familiar to the user.	What was your experience of the readability and language in the app? How was your understanding of the characters appearance in relation to their descriptions? What was your understanding of the intention of the application? Did you get a sense of success when completing the given tasks?
3.	User control and freedom: user should be able to move freely in the application, and there should always be a way back.	How did you experience exiting events and returning to a previous page in the application? Did you feel a sense of control when navigating the application?
4.	Consistency and standards: the user should never feel confused over	Did any terms, labels, or icons feel unfamiliar or confusing to you? Which?

	certain words or actions, typically the developer follows industry standards.	Did you recognise any visual elements from other mobile apps? Which? Have you experienced similar apps or websites to this?
5.	Error prevention: it is important to keep the user informed if an error occurs, good error messengers are important	<i>This principle was unachievable to implement as the prototype was unable to inform users of errors.</i>
6.	Recognition rather than recall: it is important to make recognisable actions rather than making the user learn new interactions.	How was your experience finding the different features of the application? How was your experience with the navigation bar? Did you feel the need to learn something new in order to navigate the application?
7.	Efficient use: it can be efficient to have shortcuts to save users' time	Did you experience a feeling of inefficiency when navigating the application? Did you experience obstacles to complete the given tasks? Which and how come?
8.	Aesthetic and minimalistic design: help the user keep focus on the main functional elements of the application.	Did you experience any visual elements that drew unnecessary attention away from the given tasks? How was your experience of the visual content and design?
9.	Help users recover from errors: help the user return to the application after a possible error.	<i>This principle was unachievable to implement as the prototype was unable to inform users of errors.</i>
10.	Help and documentations: if the user needs to perform a bigger task they need more information, this should be presented in the right context	Did you experience the lack of explanation when using features of the application? Which?

Table 4 Heuristic principles and formulated questions

The left side of the table contains the ten heuristics. The right side contains inspired questions we formulated based on the principles and later used to interview the participants, While it is

technically possible to simulate error messages in Figma (Figma, n.d.), the only potential error in our prototype was related to user login information, which Figma does not support due to its lack of real input validation. Because of this limitation, no error messages were implemented in the prototype, and therefore, heuristics 5 and 9 had no corresponding questions.

To get an understanding of how users would experience the prototype, we conducted a usability test with four students from the same group of participants of our previous workshop. This time, the group consisted of three men and one woman. Three participants were international students, and one was Danish. By conducting the test, we aimed to gather relevant insights by observing our target group in a natural setting, focusing on feedback related to functions, issues, or elements in the prototype that users might find confusing or unclear, or that we as designers might have overlooked during the development process. The mix of international and Danish students was intentional. The international participants had recently relocated to Aalborg and were still becoming familiar with the city and local cultural norms around the different types of events. The Danish student offered insights with cultural familiarity, while still bringing the perspective of someone newly settled in Aalborg. This combination allowed us to evaluate the prototype's usability across diverse user backgrounds and levels of familiarity with the city.

The usability test was conducted at Aalborg University's central campus, CREATE, where all participants were currently studying. The setting was chosen to help provide a natural and familiar environment, aiming to help the participants feel more at ease. To ensure focused feedback and avoid participants influencing each other's responses, each participant was paired with a group member, and the tests were conducted individually.

Before the test, we had defined three tasks that each participant had to complete in the prototype. These tasks consisted of:

1. **Your first task is to match with the character you want to match with the most.**
2. **Your second task is to see when an event is taking place.**
3. **Your third task is to add an event to your calendar.**

The purpose of these tasks was to help us gain insights into our problem domain through user observation. During the test sessions, we took notes and encouraged participants to "think out loud", hoping to get an understanding of their thought processes as they navigated the prototype and completed the tasks. We aimed to refrain from interfering unless necessary, in order to observe their intuitive approaches to solving the tasks. Once all the tasks were completed, we

asked follow-up questions to gain further insights into their experiences of the prototype. These included the heuristics-inspired questions from Table 4, as well as any spontaneous questions that emerged when observing the tasks being completed. Each group member took on the roles of observer, note-taker, and interviewer during the usability tests, which led to variation in the level of detail and structure in the data we collected. While this approach lacked the consistency of a standard testing procedure, it allowed us to have a data collection process that captured a wide range of the participants' reactions and user experiences. Despite the differences in the individual test sessions, several common themes emerged in the response to the prototype, which gave us a clear indication of which elements and functions of the prototype required further refinement. The data gathered from the usability tests can be accessed in Appendix 2

3.2.4.2 Usability Evaluation: Severity Analysis

To organise the data gathered from the usability tests, we conducted a severity analysis. We applied the questions from Table 4 (See section 2.4.3) to structure and interpret the gathered usability feedback. The table below (See Table 5) presents a question asked during the tests, relevant participant responses, and the corresponding suggested changes to the prototype based on the severity of the problem. As a group, we discussed which changes were the most relevant and realistic to implement within our current skills based on our pre-existing system requirements. For instance, if a user had difficulties navigating the interface or expressed frustration when selecting a character, that would indicate a more severe issue and would then have a higher priority.

To help prioritise the design improvements, a severity rating was conducted based on user feedback and how problematic the user found the given task. The severity of each problem was evaluated using three key factors: the frequency (how often the problem occurs across users), impact (the extent of the problem) and persistence (whether it is a temporary or consistent problem). (Barnum, 2021, p. 308) These were categorised into four degrees:

Cosmetic: The problem is hardly mentioned and is therefore only a priority once other, more valuable tasks are completed.

Minor: The problem needs fixing before release, but does not significantly hinder or affect the user's experience. It causes minor inconveniences and frustrations and should be given low priority.

Major: Is for more prominent problems that noticeably impact on the user experience and is given high priority.

Catastrophe: Problems so big that they hinder the progress of further development. Imperative to fix before anything else (Nielsen, n.d.).

In the table below, the questions and responses from the usability test are presented in the left and with the derived design additions and their corresponding severity rating, ranked from highest- to lowest priority.

Question	Response	Design Addition and Severity Rating
What was your experience like when exploring the characters /calendar / timeline / profile?	User 1: experienced an overall sense of joy and ease, but expressed confusion regarding the “home” button and page. User 2: Some aspects such as the back button, moved to different locations depending on where he was on the application. User 3: Thought of the characters secondarily and the events first and foremost. “Matched” with actions over characters. User 4: “She did not catch the personas from the first page of the app; she understood them better from the map and timeline page, where the names are related to their events. She liked the characters, but the brainy character needs a new name. There is missing information about them.”	Major: The users had difficulties tying the icons to their content. We need to create a correlation between users' predicted interactions and actual outcomes when interacting with the application. Major: The names of the characters should reflect the category of activities they represent when introduced to users for the first time. Minor: The users could resonate with several characters, so a questionnaire, when creating a profile, could help guide the users in matching with the “correct character”.

What was your understanding of the intention of the application?	User 1: understood the intention of finding events but disapproved the idea of “matching” with a specific character, as he related to several of them. User 2: it was difficult for the user to comprehend different aspects of the Calendar function, as the test wasn't carried out on his mobile device.	Major: The text and imagery in the app should translate the categories of events and locations more, instead of focusing on the characters and their personalities. Cosmetic: The words “vibe” and “match” insinuate a relationship to the characters instead of the events and locations – there should be more cohesion between the language and the information provided to the user.
Did you feel a sense of control when navigating the application?	User 1: expressed dissatisfaction with the placement of exit buttons, as he had to look for them instead of them being “..where they usually are.”. User 2: The user expressed a sense off confusion when “matching” with his persona, he felt like losing control and didn't understand if the event a guide from the character or his personality should match said character. User 4: “You know that the exit button always is in the top right.”	Catastrophe: The exit and return buttons should follow industry standards and be placed in either the right or left top corners. Catastrophe: The user should not feel like they are “put in a box” when choosing a character, so there should be less focus on the character they choose and rather on the category of activities they prefer. Major: The user should be able to access a “help” button or “support” feature when experiencing a lack of control using the application.
Did any terms, labels, or icons feel unfamiliar or confusing to you? Which?	User 1: The user recognised all of the navigation icons but had difficulties connecting the character designs to their categories. User 2: The user was tasked with picking a character that matched his “vibe”. The wording in this case could use some tweaking.	Cosmetic: The design of the characters should be cohesive with their categories, but should not be more important than the activities they contain. Minor: There should be consistency in the wording to avoid the user feeling confused. Cosmetic: It could be an advantage to encourage users to participate in events

	User 3: The user said that Individual activities should show if it's 1on1 or a group activity. They would like to use the profile to save their interests and get recommendations.	if they could see others who plan to participate. Major: The characters should be neutral in gender expression to avoid a gendered bias.
Did you feel the need to learn something new in order to navigate the application?	User 1: The user expressed difficulties navigating and understanding the features at first, but after a few minutes, he expressed a feeling of ease navigating the application. User 2: The user expressed that although the characters gave a recognisable feeling to the application and navigation tools within, the process of familiarising with the characters where a process of its own.	Major: The interface design should have increased focus on the affordance to create more intuitive navigation in the application Major: The character descriptions should be shorter to avoid the users having a cognitive overload of information. Major: The navigation menu should highlight which page the user is currently viewing to help users navigate the application and increase their sense of control. Major: The application should have an option to read more about the characters without "selecting" them.
Did you experience obstacles to complete the given tasks? Which and how come?	User 1: The user experienced difficulties finding the "add to calendar" feature. User 2: The user needed an overview of the events, so they didn't need to scroll through all characters to see which events were connected to which character. User 3: The user suggested keywords explaining the activities would help choose a character. User 4: The user didn't like that you could not swipe between the characters.	Catastrophe: There should be an add to calendar feature on all events, both in the timeline and calendar pages. Minor: There should be a shortcut to the character cards to easier access the list of their favourite events and locations Cosmetic: Swipe functions could be implemented as an intuitive way of navigating the application.

Table 3: Severity scale based on user feedback

How was your experience of the visual content and design?	User 1: The user found the visual content fun and joyful but expressed a lack of cohesion between the style and personalities of the characters to the content and the seeming target group of the application. User 2: The interface and navigable elements made sense, but the theme drew too much attention from the usability; it felt a bit too childish. User 4: The user felt like when viewing the timeline page, when you press on an event, the map and location of the event should be shown instead of where you are on the timeline.	Major: The colour scheme of the application should be less saturated to avoid correlations to childlike cartoons as well as gender stereotypes. Major: The presentation of an event should include the date and address. Cosmetic: The size of the characters needs to be smaller to avoid drawing attention from the displayed events.
Did you experience a lack of explanation when using features of the application? Which?	User 1: The user had trouble finding back to the timeline and needed guidance to return to it. The user also had trouble differentiating between the characters' personalities as "...they all aimed to chill, so what's the difference?". User 2: The user expressed a need for a tutorial when they first encountered the characters, because it was not clear what the application was about or why the characters were there. User 4: The user felt like it was difficult to find out what the characters represented in terms of event and activities.	Catastrophe: The application should include contextual hints to the contents of the application. Catastrophe: The title on the character cards needs to be based on the category of activities instead of the characters names. Catastrophe: The application should have a tutorial the first-time users open the application. Major: The description should also include listed locations with activities that fit each character.

By applying this framework, we were able to systematically prioritise design revisions according to user needs. The next section outlines how each of these issues informed concrete changes implemented in the second iteration of the prototype.

3.2.5 Changes in the Second Iteration Prototype

In this section of the report, we outline how the design additions were implemented in the second iteration of the prototype, based on insights from our analysis and extensive research on UX/UI principles. We also explain how these changes improve usability and clarity for the target group and include visual representations of the updated prototype. The terms *affordance* and *natural mapping* have been briefly mentioned earlier in the report, but the redesign of the application drew largely on further implementation of the concepts.

Affordance refers to the properties of an object that imply how the object can be used to carry out an action. In the context of Interaction Design, affordance is used to guide users on what they can do without having instructions (Helen Sharp et al., 2019, p. 30). An example in the context of our prototype is the placement and appearance of navigation buttons throughout the application. Users know what buttons can be pressed because they resemble buttons they have pressed in other applications or even real-life encounters. Affordances provide useful visual cues that can help users understand the tasks they carry out within an application (Helen Sharp et al., 2019, p. 30).

A related key concept in user interface and interactive design is *natural mapping*, stemming from Norman's second heuristic design principle on stimulus-response compatibility in the interface design. This means the placement of the controls and buttons in an interface must correspond to users' expectations in order to be useful. The icons should represent actions, such as a home icon that takes users to the "home" of the application. The language and layout should speak to the users' understanding of how the world is built, so even if a user has never seen a page before, there should be enough conventions in place that the navigation is intuitive and obvious. Users should not feel frustrated or lost because they don't know automatically what to do next. (Nielsen Norman Group, n.d.)

The two concepts have similar purposes, but affordance points to what actions are *possible* and natural mapping refers to how actions *match outcomes*. Having this in mind, the implementation of the design additions in the second prototype is not only based on the derived design additions of the severity analysis, but also on the additional knowledge from the concepts above.

3.2.5.1 Implementation of Design Additions

In this section, we describe how design additions were implemented in the second prototype and how the changes reflected the feedback and knowledge derived from the severity analysis. The changes were made across all characters, but for clarity and consistency, we will display the changes of the social character, Topsy, as a representative example.

The catastrophic usability issues identified through the severity rating table were addressed as the highest priority in the second prototype. Among these, we chose to focus on disruptive problems and resolved them by applying the principles of natural mapping and affordance. One recurring problem was the inconsistent placement of the return arrows, which users found confusing and frustrating (see Figure 13, 1.H and 1.I). The return arrows appeared in different positions across pages, which disrupted users' expected flow and went against the principle of natural mapping. This inconsistency led to users feeling frustrated and confused, and the design addition was rated as Catastrophic. To resolve this, we removed the return arrows entirely and instead implemented an exit button (X) located in the top-right corner.

In the second prototype, we have deprioritised the character aspect by changing the titles on each character card to reflect their category of activities instead. For example, the character Topsy was originally based on the "social" category, but this was only visible in the headline at the top of the app. (See Figure 12, 1.D) In the second prototype, the card is titled *Social*, with the subtitle *Events and Locations*. The card also includes a shorter description with examples of the types of events Topsy would typically go to. (See Figure 11, 1.C, 2.C) This change was based on feedback we received about confusion over whether the characters were meant to be relatable or more like fictional users to "follow". To clarify this, we shifted the focus toward the types of events and locations that match the characters' personalities, and therefore, the users' interests.

Some users described the look of the characters and colour scheme of the prototype as childlike, which strayed away from the intended target group. Users also expressed that some characters came across as strongly feminine, masculine, or androgynous, which caused users to choose characters with a corresponding gender to their own. We took this into account and adjusted the design of the characters, aiming to remove gender connotations and the colours of the characters. The overall colour scheme was also toned down. (See Figure 11, 1.C and 2.C).

When using the application, users were not sure of the contents and intended use of the application. When faced with the login page, several users expressed confusion about what

they were logging into as well as general contextual clues throughout the application. To accommodate this, we added a map of Aalborg to two of the pages within the application. The first one being on the login page, which in the second prototype hints to the contents of the application through the pictured map and listed categories of activities (See Figure 11, 1. A, 2.A). We also added a map to the pages when events are displayed, showcasing the distinct location where a selected event would take place.

An addition in the second prototype was related to a task the users were given, which consisted of adding an event to the calendar. In the first prototype, the option to add an event to the calendar was only available if the event was selected in the calendar page of the application. The problem here was that the events were also displayed on the character timelines, and when pressed, that same option to add to the calendar was not available. (See Figure 12, 1.F, 2.F) The problem could be resolved by adding the missing button on the events from the timeline.

An important thing to acknowledge is the catastrophe-rated design addition that was not implemented in the second prototype. The identified issue was the lack of guidance when users first navigated the application and interacted with the characters. Several of the users expressed confusion and a need for a tutorial or a “help” option. While we acknowledged the issue's severity and rated it as a catastrophe, we chose not to implement a tutorial in the second prototype. Although it would be technically possible to implement in Figma and the coded prototype of the application, we instead chose to prioritise the design additions that would reduce the need for a tutorial altogether. Some of these changes included improvements in the consistency of navigation elements by strengthening the visual affordances. By implementing all the additions above, our goal was to make the interface more intuitive by default, allowing the users to understand the application better without additional explanation. To increase the users' sense of control when using the application, we implemented an option to easily switch between the different categories of activities by selecting a different one in the top header of the application. The categories can also be changed by pressing the arrows on the sides of the interface or swiping right or left when viewing the timelines. We also implemented a visible distinction in the navigation menu by highlighting which page the users were currently viewing. This played into the affordance and natural mapping terms, as users would be able to see where they were “located” but also that they had the option to view other functions within the application. (See Figure 12, 2.D, 2.G, 2.F).

The first prototype placed a lot of focus on the characters, giving users a backstory on their personalities and inviting them to *Match* with a character of their choosing. An example of change in the application is evident in the adjustment of language, where words like “vibe” and “match” were either changed to “select” or removed entirely (See Figure 11, 1.C, 2.C). This was done to establish the notion of cohesion and remove unnecessary cognitive overload or confusion for the user. We also added an option if users wanted to learn more about the characters by making a “bio” for each character available. The “bio” was only accessible if users pressed on the characters in their timeline pages, which enabled us to keep the information on the characters available but not forced on the users (See Figure 12, 2.F).

The severity-rated and implemented design additions to the second prototype helped us allocate time to important aspects of the technical implementation of the design. In the following section, we focus on the technical development of the application, showcasing the system development behind the final implemented prototype of the application.

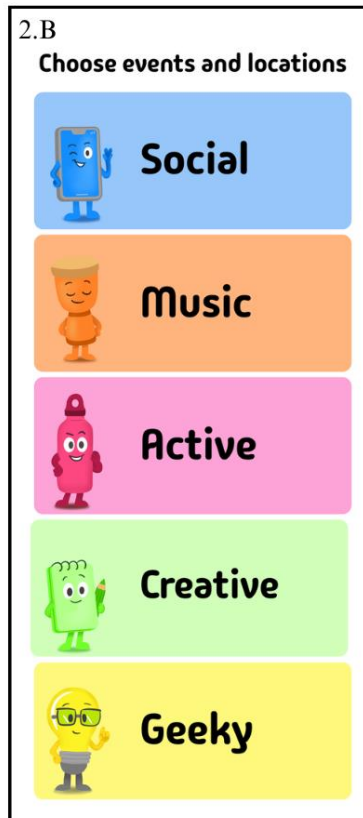
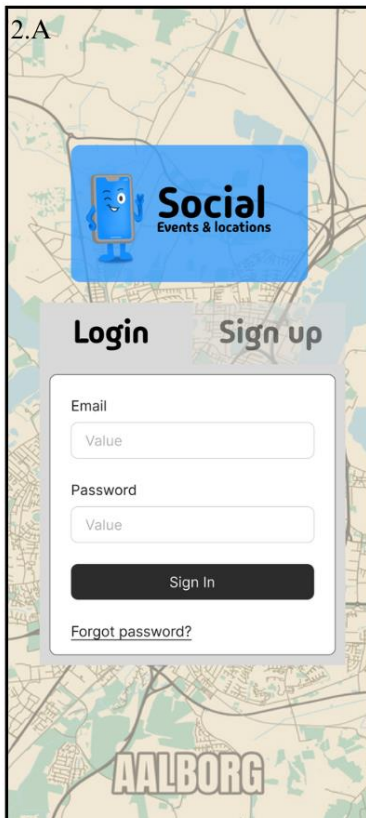
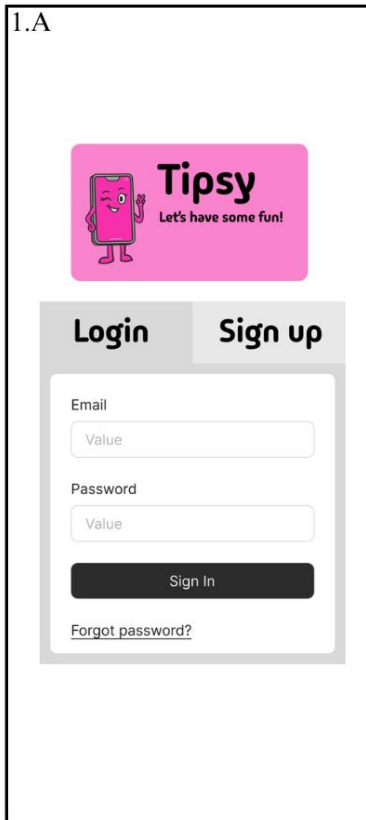


Figure 11 Comparison of first and second iteration prototype 1.A-1.C to 2.A-2.C

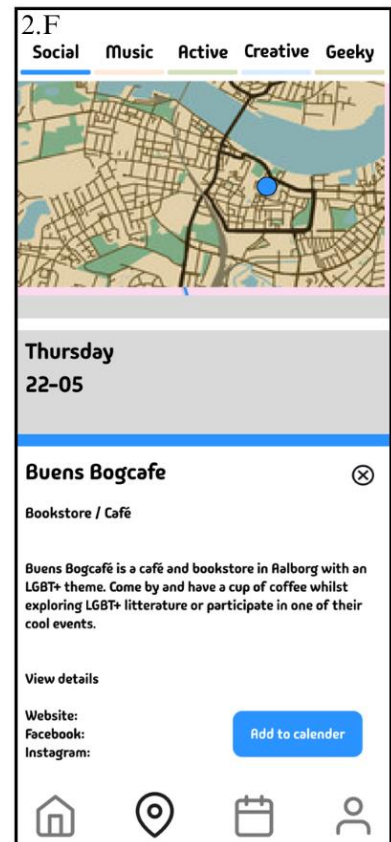
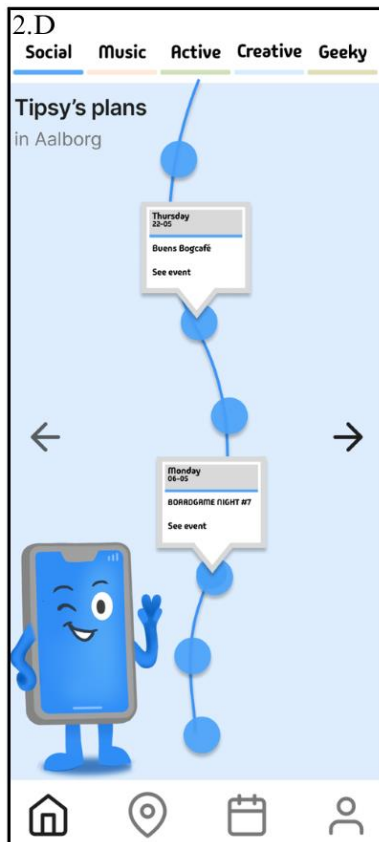
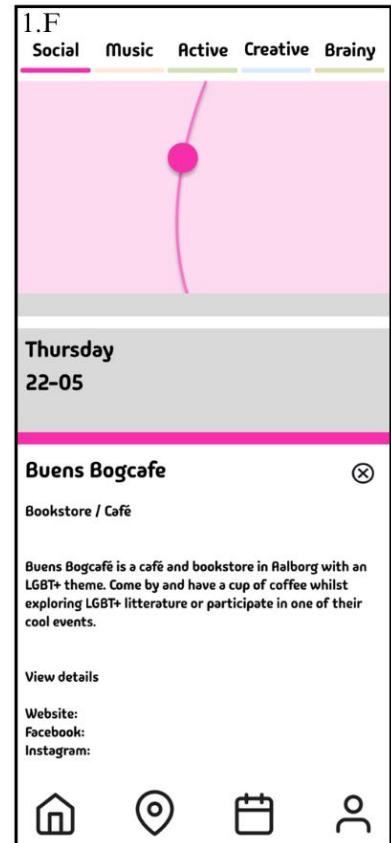
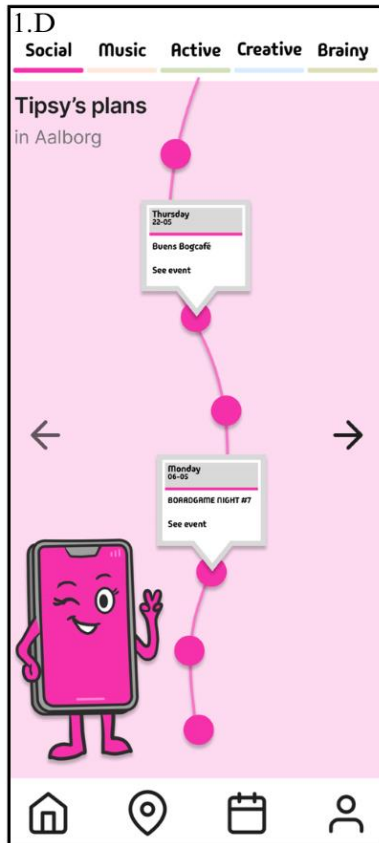


Figure 12 Comparison of first and second iteration prototype 1.D-1.F to 2.D-2F

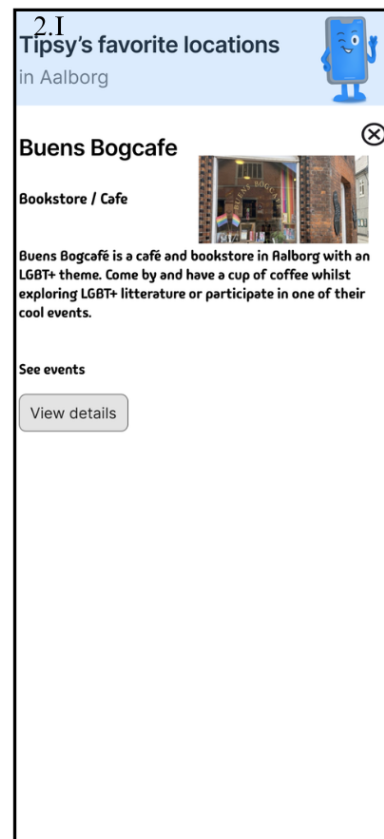
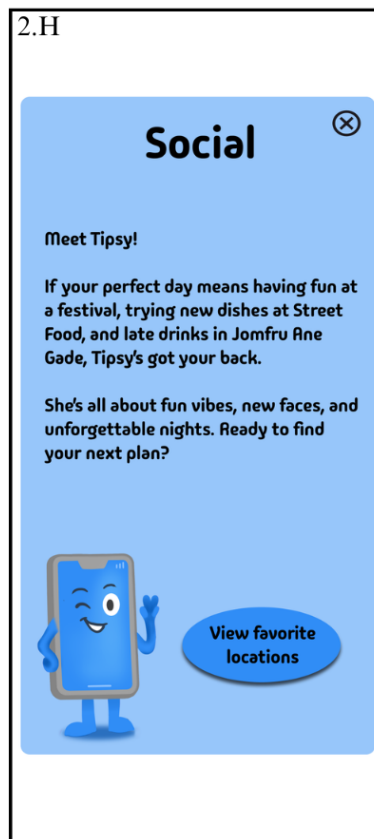
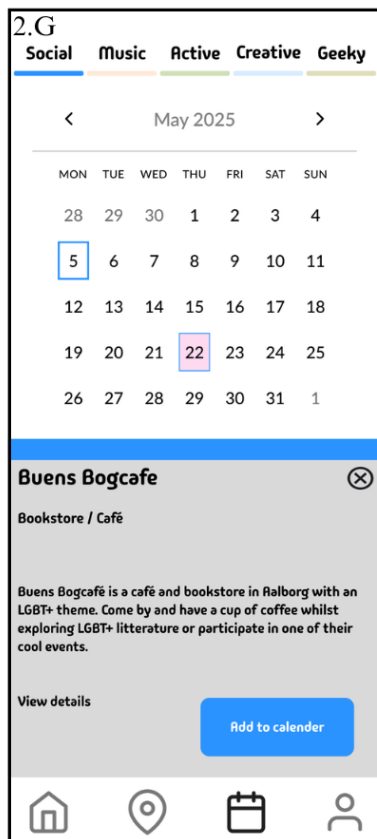
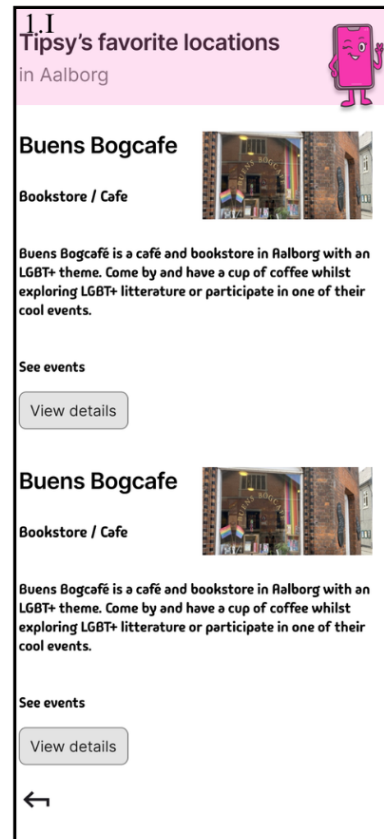
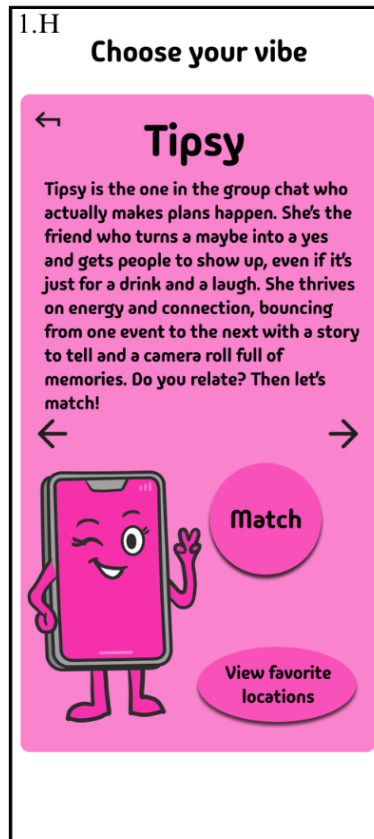


Figure 13 Comparison of first and second iteration prototype 1.G-1.I to 2.G-2.I

3.3 System Development

The following section describes the technical development of the application, showing how initial architectural frameworks and design plans were developed through the application of object-oriented programming. Class diagrams and behaviour models were created based on earlier system requirements with the means to define the structure, logic, and the interactions within the app. The development process used a layered architecture to keep the interface, function, and model layers of the system separate and easier to manage. Implemented in Android Studio using Java and XML, the system was developed, with components like user profiles, characters, and events to support a personalised user experience. This section illustrates how these technical elements were shaped by the projects design goals and how they were integrated into a cohesive application architecture.

3.3.1 Object-Oriented Programming

Object-Oriented Programming (OOP) is a central aspect of this project, as it is the paradigm used to develop our application in Android Studio, using Java. At the core of OOP is the concept of the *object*, a modular unit that can contain variables and subroutines. One of the primary motivations for using OOP is that objects can be constructed in a manner which allows them to more accurately represent real-world entities, compared to other programming paradigms. Objects are organised into *classes*, where encapsulated information within subroutines contributes to the flexibility of functionality (*Introduction to Programming Using Java, JavaFX Edition*, n.d.).

A class can have *subclasses*, each containing relevant subroutines, all of which inherit functionality from the parent class. Classes can be designed to represent real-world entities and are composed of attributes that define the behaviour and purpose of the object. A key advantage of OOP is the ability to reuse classes and subroutines across different projects, as they maintain consistent behaviour regardless of context (*Introduction to Programming Using Java, JavaFX Edition*, n.d.).

Each object in OOP has its own set of responsibilities and can hold distinct data. Consequently, objects may respond differently to the same message, and they interact through message passing. We adopt OOP for its bottom-up logic, which enables us to reuse classes and their functions at various points within the application's structure, and it allows us to model objects

in a way that closely reflects their real-world counterparts (*Introduction to Programming Using Java, JavaFX Edition*, n.d.).

3.3.2 Design of our application

Our sections in our system development are separated into two parts. First part is about the design of our application, which clarifies how we have assembled the classes and architecture of our application design, with the goal of showing the schematic to utilise the benefits of object-oriented programming. Due to how this is a second semester project, and the knowledge barriers involved, we have not programmed a prototype application that includes all the features and functionalities that is included in our application domain (Mathiassen et al., 2018). Our prototype instead focuses on the important and unique parts of our application. Planning and including the complete design helped program the prototype. The included classes and functionality are made to behave as if the rest of the program functioned as designed. In the later part of our System Development section, we make it clear which parts of the application were made for the prototype, how they are made- and work by going through the code, and how the less important parts of the application prototype were made visually but lack functionality.

Moving from the prototype to the development of the application, it is important to organise and document the application in various ways. In this section, we go through the different diagrams we made to plan the structure of the application. These diagrams made the construction of our application easier, as it helped coordination between group members, which shortened the time spent programming.

3.3.2.1 Event table

In this section, we examined the problem domain through system development. We specifically identify the relation between classes and events, using a simple event table (See Table 6). The table maps event (e.g. sign up, event chosen) to the relevant classes: Student, Venue, Event, Interest types and Organisation.

The symbols + and * are used to indicate the involvement for each class to the corresponding event. + indicates that an event can occur zero or one time. * indicates that it can occur zero or more times (Mathiassen et al., 2018).

Classes / Events	Student	Venue	Event	Interest types	Organisation
Sign up	+				
Participating in event	*				
Event planned					+
Event chosen	+			+	
Venue chosen	+	+	+		
Choose favourite venue				+	

Table 6 Event table for Problem domain of project

The Events class, as represented in the table, is an object that can be interacted with by the Student, Organisation, and Venue classes. This mapping establishes the relationship between events and problem domain classes, facilitating a clear understanding of which entities are involved in specific actions within the system. The Venue class, as indicated in the table, is a physical entity, representing specific locations where events occur, rather than an active participant in system events. The Interest Types class, on the other hand, represents the abstract area of student preferences. This class was derived from the persona method. The method is used in the earlier stages of the project to capture detailed user profiles based on interests and behaviours. Interest Types are used to model specific student preferences that guide event and venue selection. This concept later evolves into the Character class in the application domain, which will be explored in greater detail in the next section.

3.3.2.2 Class Diagram

Moving into the application domain and building the application with Object Oriented Programming, it was important to have a plan for what classes we would use and how they fit together. The first diagram includes the main classes that founded the model layer and the functionality layer of our program. The “behind the scenes” classes structure and work with the data our application uses. The class diagram illustrates elements extracted from the problem domain: events, venue and character and how they work together with the elements created through the application domain, Calendar, Timeline, Map of event. The character class is the combination of the interest of students identified through the problem domain and the personas method, mentioned in section 3.2.2.

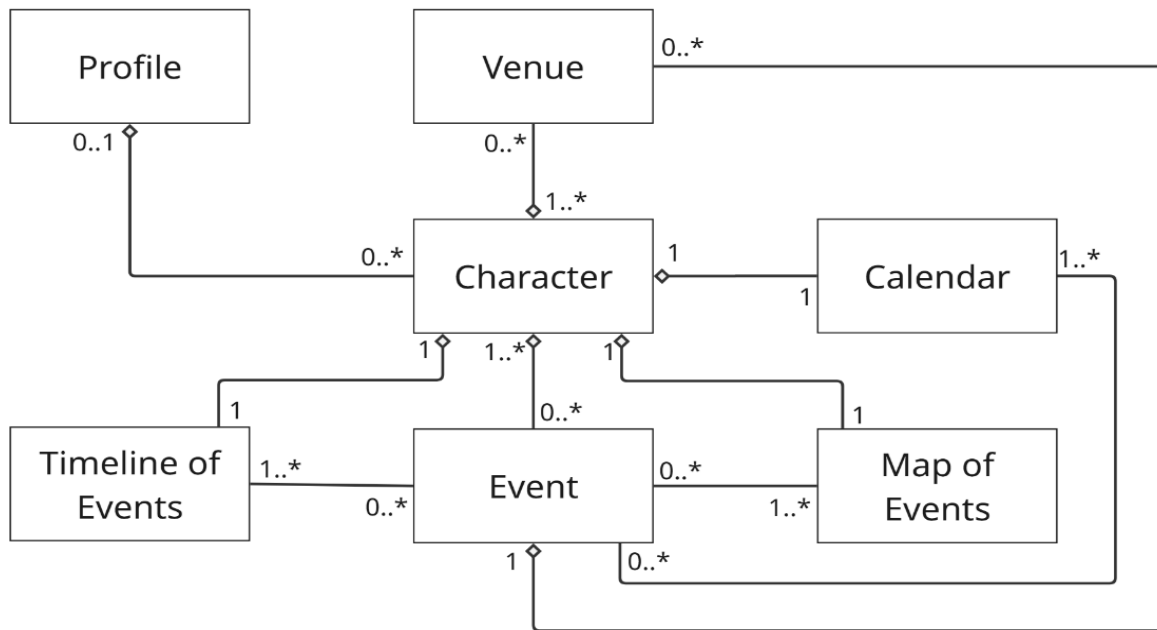


Figure 14 Class Diagram that shows how the different classes in our model and function

Most classes in Figure 14 are connected by arrows with white diamond heads. These arrows signify a “has-a” association, also called *aggregation*. This means that an object of the class, the arrow points to, “has an” object of the class the arrow points from. The objects that the arrows point to are called the *superior* objects, and the objects which the arrows point from are called the *inferior* objects (GeeksforGeeks, n.d. In the case of the Profile class and the Character class, Profile is the superior class because it contains a Character. The numbers by the arrows show how many inferior objects the superior object is composed of, and vice versa. When there are two numbers, they show the lower and upper bound respectively. When one of those numbers is an asterisk, it says that there is not a defined limit. In the example between Profile and Character, the number by the Profile demonstrates that one profile object has between zero- and one-character objects, and each character can be chosen by any number of profiles. The “..” here means “between”.

We want each profile to be able to contain between zero and one characters, since there is a period of time after a profile is created before the user has chosen a character for the profile. Aggregation is also useful here because it allows a profile to easily switch characters, without having to edit the code of the Profile class, and more characters can be added later without having to update Profile.

The characters are the centre of our program, and every class under Profile in Figure 14 is made to structure information and functionality that each character has. The main functionality of a character is to recommend events, and therefore, each instance of the `Character` class has a list of Event objects. Each character uses their list of events when creating the `Timeline of Events`, `Map of Events`, and `Calendar`, which can be made to only show certain events from the list. Depending on criteria like only showing events that happen next week, or to show events from different lists together. These classes are shown by the line to be *associated* (Mathiassen et al., 2018, p. 339) with the `Event` class. The timeline, map, and calendar are used when displaying the character's timeline and map in the application and get the events from the list of events. Each character can use at least one event, though they should contain multiple, but each event can also be used by multiple characters. It is important that more than one character can show the same event. An event is always hosted by one venue, and each character uses a list of venues when the user asks to see their "list of favourite locations". The reason we have favoured composition as a method of connecting the classes, is to make each part more modular, making it easier to change and update any class. If the existing interface for a class is not edited, other related classes will not need to change.

3.3.2.3 Our Architecture and it's Components

We have grounded the architecture of our application on the Layered Architecture Pattern, when designing the programming. This pattern structures software into distinct layers, by decomposing a system into components (Mathiassen et al., 2018). Each layer contains separate parts of the program, where the interaction with other components in the same layer is not essential for their operation. Components within a layer can be functionally independent from each other. As an example, the Timeline View component is independent and principally works without Description View, nevertheless relies on Timeline and Character. This structure supports maintainability, reusability and scalability of the system design.

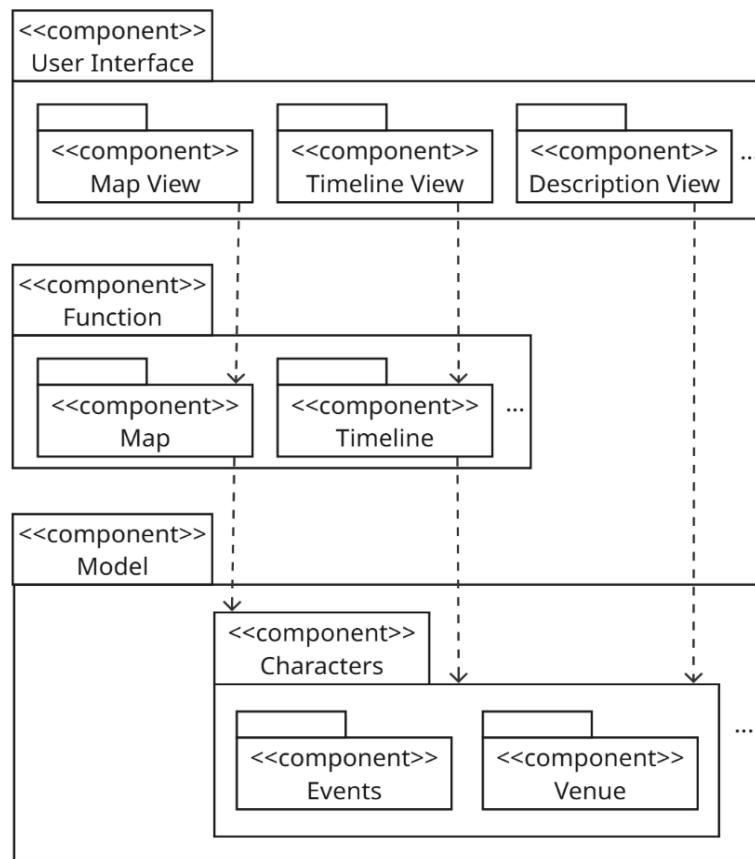


Figure 15 layered architecture pattern of our application

Figure 15 gives an overview of our system's layered architecture and illustrates core components interact across layers. Our diagram has three layers, Model, Function and User Interface. The bottom layer, called the *Model* component, contains the components which is derived from the problem domain. We have placed the characters in the model component, and consider them closely related to the problem domain, because they are abstractions of us as students in Aalborg, and their recommendations reflect our own experiences. What kinds of functionality each character contains is also expected to be largely static, which is expected of components in the model layer. The *function* layer and the *user interface* layer are the components more separated from our problem domain. It is their goal to administer the model component and allow the user to interact with it (Mathiassen et al., 2018).

Within Figure 15, the User Interface component consists of three views: Map View, Timeline View, and Description View. These indicate three pages of our application and are examples of how they use our character class. The Description View depends on the character component directly for the text and colours it displays, while the Timeline View and Map View are dependent on their respective Function components to manage and sort through the events and venues that comes from the Character component.

The focus of the diagram is on the different roles of the Characters component and how the different layers use it, therefore, we chose not to include the database consisting of the profiles, characters, venues, and events in the model. The model component contains the profile component, but the diagram does not include it as seen in diagram 2 Figure 15.

Our architecture uses the *open-strict* variant (Mathiassen et al., 2018), which specifies how each layer and its components can rely on each other. Being open means that components in any layer can rely on components from any other layer, as opposed to only allowing dependency between layers directly above or below each other. It has the advantage of allowing our design to skip what would otherwise be needed. Pointless components of the function layer, like a component whose only functionality is to pass a character description and other info from the character component to the description view component, is non-essential. We are aware that a closed architecture can help limit how many components interact directly with the model layer, thus making the model easier to change, but as there are not many layers in our architecture, we favour the flexibility from it being open. The strict descriptor means that dependencies can only go from an above layer to a below layer. Since we expect the interface layer to change more regularly than the function layer, which changes more frequently than the model layer, we want any component to depend on more static components, therefore, we chose the strict architecture pattern.

The application domain is generally going to change more often than the problem domain (Mathiassen et al., 2018) This should also be the case with our application, as its interface and presentation of information would probably need updating more often. Any dependency makes it more difficult and time consuming to make changes to a component's interface, as any dependent components would also need to change. That is why it makes it easier to update the application, if we limit dependencies on components that will update an equal or less frequent amount. Our chosen open-strict layered architecture enables us to balance flexibility and maintainability. Its use promotes a separation between stable domain concepts and changing user interaction logic, allowing us to develop a scalable and adaptable application while ensuring long-term robustness (Mathiassen et al., 2018).

3.3.2.4 Flowchart of user interaction

The flowchart in Figure 16 demonstrates the timeline of the events. The purpose of this diagram is to show the user interaction of choosing a category in the ideal finished application. This is used as a guide for the programming of selecting a category.

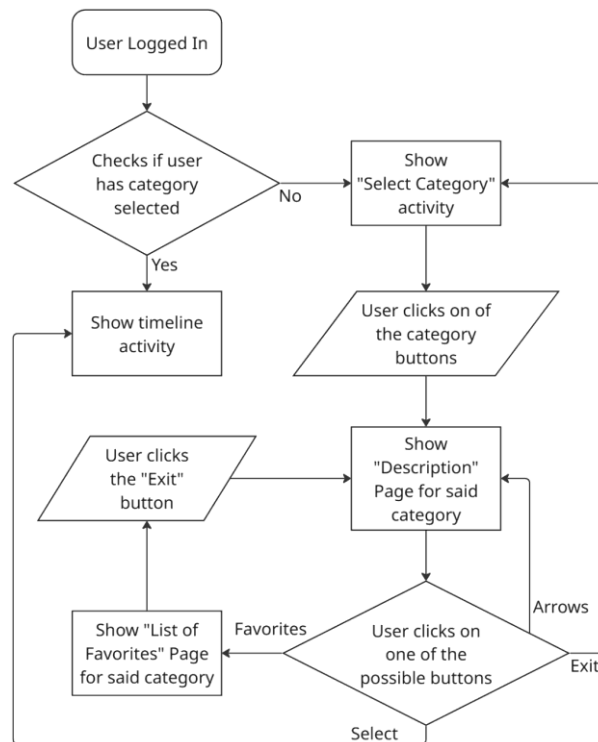


Figure 16 Flowchart of user choosing a category

After logging in, the app first checks if the user already has a character selected. An existing user will have one selected and be taken to their activity timeline directly, but a new user will have to choose a character. This starts with the screen displaying the five categories, where the names are retrieved from the character objects. After choosing one, the user is shown a more in-depth description, also with info taken from the chosen character, where there are many possible buttons to press. The arrows on each side of the screen take the user between description pages for each character. These description pages and the “Select Category” page, reference a list of all the characters included in the app, in order for the user to dynamically cycle through each available character. The user can also exchange between the description page and the list of said character’s favourites. Exit the page back to the overall selection screen and select a category in order to be taken to the same corresponding activity timeline as an existing user.

These system development diagrams serve as a blueprint for the implementation phase of the application. They help bridge the gap between the design and actual coding by providing a clear structure for the system's architecture (Mathiassen et al., 2018). These diagrams support the development process by translating the conceptual design into a concrete foundation for coding, helping ensure consistency between what was planned and what is built.

In the subsequent section, we will explore how the design translates into working code, outlining the specific features implemented in the prototype, discussing how they were programmed, and highlighting the differences between the designed and implemented versions of the system.

4 Implementation of our Prototype

As stated earlier in the chapter, this is the part of the system development chapter which goes over the details of the prototype we have programmed for our application. We will go into details on how certain parts of our application design were implemented in our prototype.

4.1 Which parts of our design were implemented?

For this project we made a T shaped prototype based on our design of our application domain. “A “T” prototype combines advantages of both the horizontal and vertical prototypes [...] offering a good compromise for design evaluation.” (Hartson & Pyla, 2019). In our prototype this means that we have implemented the broad features involved in the program, e.g., creating the overall layout for buttons, but only creating functional buttons on certain features. In our prototype, we implemented elements in both the horizontal and vertical.

The horizontal bar of the prototype is represented through the layout design, which includes navigation elements for all major features, even those that are not yet fully implemented. We created a fake database to simulate backend functionality. This allows us to mimic a real database and how it would support features such as events, users, and venues, which enables interaction at an early stage. The vertical bar focuses on selected features developed in a greater functional depth. This is on the character creation and management features, allowing users to select and view detailed character descriptions. We also developed the timeline and calendar functionalities, where users can view events associated with the characters, and for the calendar, specifically manage the events over time. Additionally, event and venue creation are partially implemented to demonstrate how they will, over time interact with character timeline and system structure.

This selective implementation establishes an overview of the system (the horizontal bar), while supporting user experience through enough detail and functional features (the vertical bar). This structure enables future vertical development, building on the horizontal foundation already established. This approach necessitates a clear understanding of the structure of the application, in which system development becomes important. System development described in section 3.3 worked as the foundation in selecting the features for the “T” structure of the application. We used the general design principle of “Favour composition over inheritance“,

thus we make sure to reuse code. This practically means we create classes that include instances of other classes (*GeeksforGeeks*, n.d). This is the “Has-a” relationship, which can be seen in Figure 14. A concrete example of this could be how the `Character` class “has-an” array of events and venues, which are connected to the `event` and `venue` class.

```
10 public class Characters {  
11  
12     // Other attributes  
    6 usages  
13     private ArrayList<Event> events; // Ensure Event class is properly imported  
    4 usages  
14     private ArrayList<Venue> venues; // Ensure Venue class is properly imported
```

Figure 17 code snippet showing "Has-A" relationship

To effectively implement our prototype and apply the design principles described above, we needed a development environment that aligned with our technical and practical requirements. To get a feel for how the final product was visualised, see Figure 18 below. The figure is used throughout the implementation as a reference point.

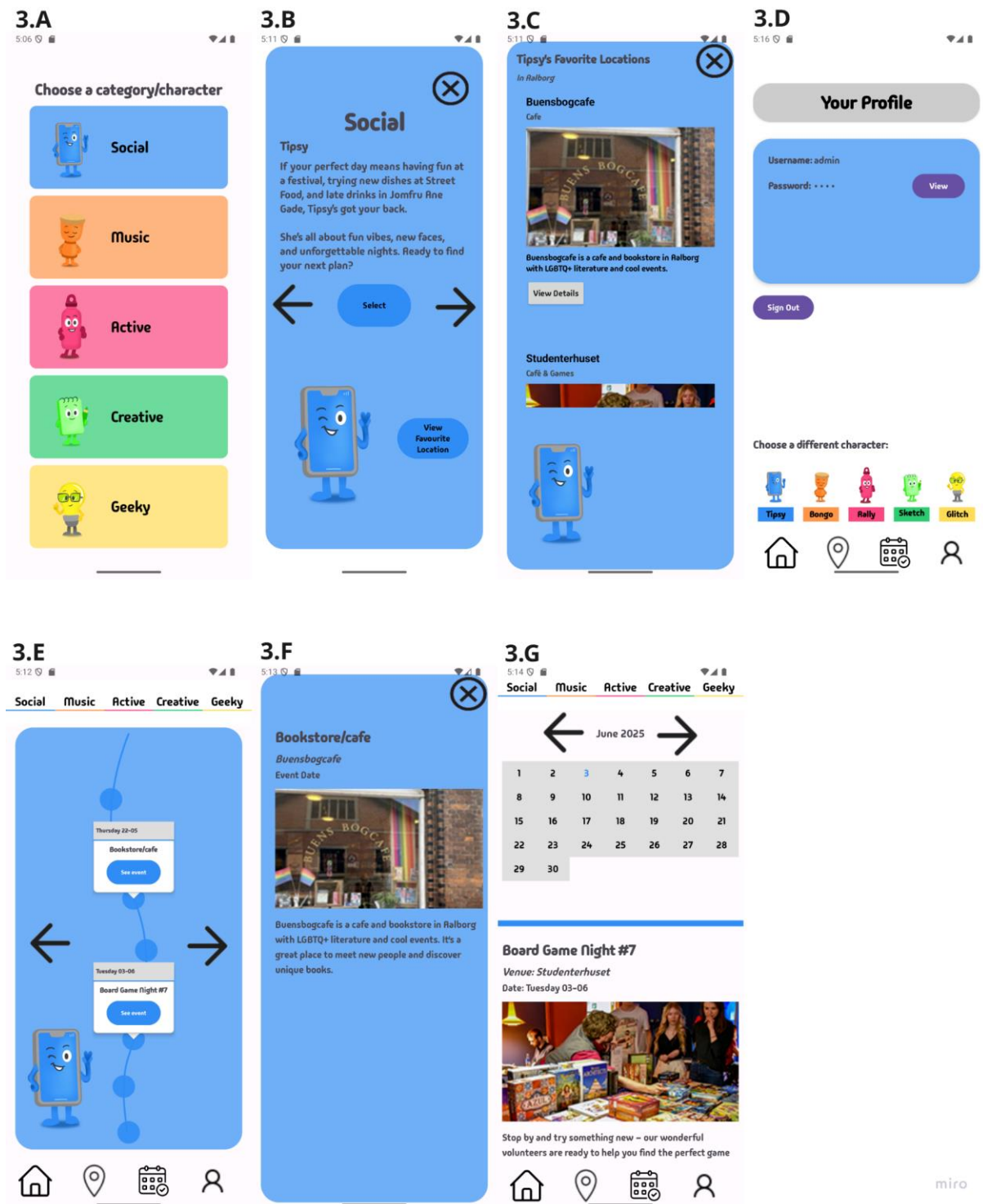


Figure 18 Final Application Prototype

4.2 Applications in Android

Android Studio is the Integrated Development Environment (IDE) that we used to develop the application. We built our application on the Android platform primarily due to practical constraints: it supports development in Java, and development for iPhone (IOS) is generally more time-consuming.

An Android application is structured around activities and XML (Extendable Markup Language) files. One activity is made for each window in the application, and they manage the application's functionality, for example, specifying what should happen when a button is pressed. XML files store information about how all the elements in a scene are established and presented, where they are, and what they look like. It specifies the button's position and size. An Android app's user interface is made from **View** and **ViewGroup** elements. The **View** class is the base class similar to the superclass for GUI components in Android (*GeeksforGeeks*, n.d). The XML files can use many layouts as a base, we use **LinearLayout** and **FrameLayout**, which is a **ViewGroup** subclass. It is used to arrange *Child View* elements one by one, in a single direction based on the orientation (horizontally or vertically) (*GeeksforGeeks*, n.d, this allows us to create a layout with different elements such as texts and images).

4.2.1 Project Structure and File Organisation

The application is organised using Android Studio's standard project structure, which separates code, design, and other resources into specific folders, improving readability and maintainability. The app logic is written in Java and placed in a single package inside the **kotlin+java** directory, including activity classes that define the visual and interactive elements of each activity. The model classes and function classes manage the characters, events, and venues, as well as formatting the data and passing passed between components inside the application. These classes work together to display the content, handle user interactions, and control the navigation between the different parts of the application. The user interface is designed using XML files located in the **res/layout** folder. Each activity has its own layout file, where the reusable UI components, where the top bar and the navigation bar are defined separately, with the possibility of including them when needed. Visual elements such as the character images, icons, and custom drawable styles are stored in the **res/drawable** folder, while the font is kept in the **font** folders. These resources are referenced in both the XML layouts and the Java code, used to apply images and additional

visual elements across the application. The `AndroidManifest.xml`, located in the `manifests` directory, is responsible for registering the application's components. It declares all of the activities, defines the app's entry point, and manages permissions and metadata. The app's structure is formulated in a diagram below (See Figure 19). Note that even though the package name for our project is called `loginpage`, we named that because it was the first page we worked on, that package contains all the Java files for the project.

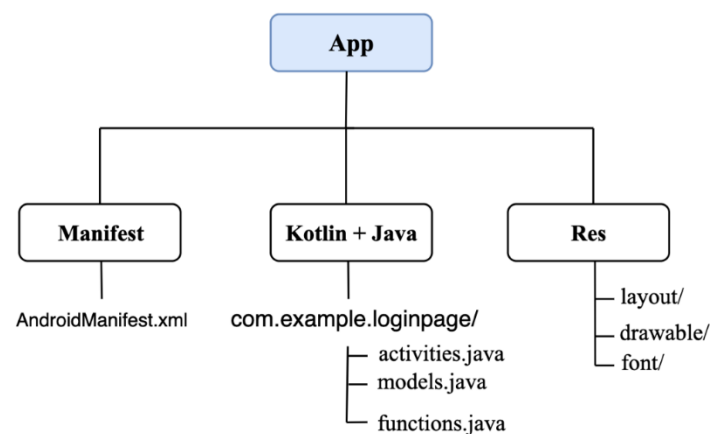


Figure 19 Detailed project folder structure

4.3 Code explanations

The following section describes the technical implementation of the main components that form our application. Through descriptive explanations, diagrams, and code snippets, we outline the different parts of the system that were developed and how they interact. Each of the following subsections focuses on a specific feature or class, including character representation, selection logic, dynamic UI rendering, event handling, and screen navigation. The intention is to provide insights into how the final design concepts were implemented through code, using Android Studio and Java. The central component illustrates how the character data is structured, displayed, and connected to the event information through a component-based development approach. Key code snippets are included to illustrate how the application was constructed and provide insights into the design choices and coding decisions made during development. The full code can be seen in Appendix 3.

4.3.1 Implementation of the characters

The `Characters` class is a core section of the application, used to represent the characters introduced earlier in the project (See section 3.2.2). Each `character` object contains all the data and the functionality that is needed to represent a character in the user interface and link them to the relevant events and venues. The class includes variables which store each character's information, such as their `name`, `category`, `description` and visual styling like `themeColour`, `backgroundColour`, and an `imageResourceId`, used in the layouts. In addition to the static data, the class has two dynamic lists: one for `events` and one for associated `venues`. These are filled when a character's timeline is created. An overview of the class's internal structure, including fields and methods, is shown in the UML diagram in Figure 20 below.

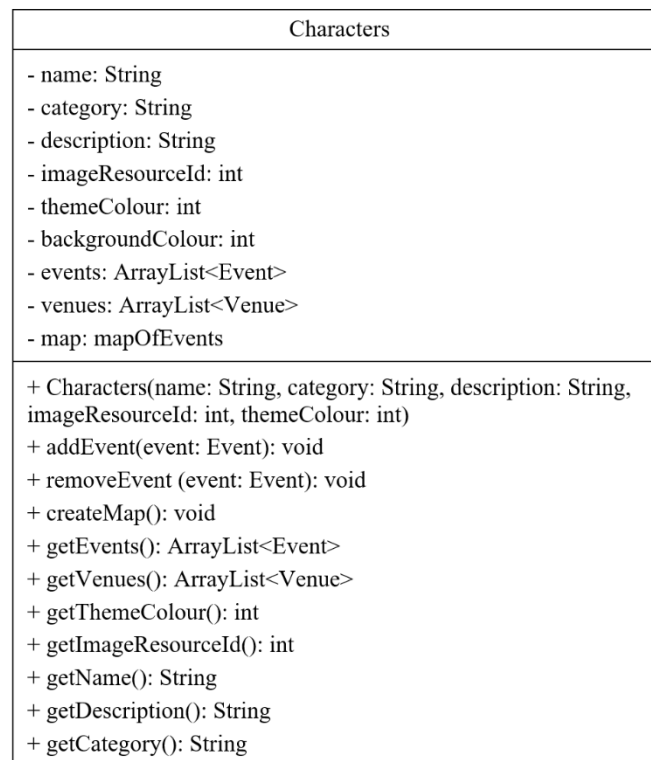


Figure 20 UML Class Diagram of the Characters Class

Two constructors are available: one with a default white background and another which allows a custom `backgroundColour`. Both of them initialize empty `ArrayLists` for future event and venue data. To facilitate user interaction and event-related content the class provides methods such as `addEvent(Event event)`. When adding an event, the method also makes

sure the selected event's venue is added if it is not already in the venue list, this can be seen below in Figure 21.

```
66      public void addEvent(Event event) {
67          events.add(event);
68          if (!venues.contains(event.getVenue())) {
69              venues.add(event.getVenue());
70          }
71      }
```

Figure 21 Method for Adding Events and Linking Venues in Characters

4.3.1.1 Dynamic Character Display

The `characterSelectionActivity` class is responsible for rendering the list of available characters and handling the user interaction. We have coded a fake database that contains the data regarding the characters. The `Character` class uses the fake database to utilise the information on the `CharacterSelectionPage`. The following code snippet illustrates how the `CharacterSelectionActivity` acquires the data from the `fakeDatabase`, as seen below in Figure 22.

```
20      // List of characters
21      1 usage
22      FakeDatabase fakeDatabase = new FakeDatabase();
23      1 usage
24      List<Characters> charactersList = fakeDatabase.getCharacterList();
```

Figure 22 Data structure of the FakeDatabase

In `fakeDatabase`, we are describing the characters with their respective information: name, category, description, image and theme colour (see Figure 18, 3.A). This is used as the data source for rendering the characters in the user interface. To access the images, we use `R.drawable`, which is a part of the Android Development resources (R) class, it provides a reference to the `res/drawable` directory. This approach provides a hardcoded, static dataset for the prototype phase while maintaining flexibility for later transition to a dynamic database.

The `onCreate()` method in `CharacterSelectionActivity` is responsible for initialising the activity's layout and showing characters at runtime. This is the beginning of the Android activity lifecycle and is overridden from the superclass.

```
24      @Override
25      protected void onCreate(Bundle savedInstanceState) {
26          super.onCreate(savedInstanceState);
27          setContentView(R.layout.activity_character_selection);
28
29          // Get the LinearLayout that will contain the character views
30          LinearLayout characterListLayout = findViewById(R.id.characterList);
31
32          // Iterate over the characters and add them to the layout
33          for (Characters character : charactersList) {
34              View characterView = createCharacterView(character); // Create a
35              characterListLayout.addView(characterView); // Add the created vi
36          }
37      }
```

Figure 23 Creating views dynamically in `onCreate()`

It sets the content view using the predefined XML Layout `activity_character_selection.xml`. The `for` loop iterates through the `charactersList` and calls the `createCharacterView()` method for each entry, which constructs and returns a custom `View` object (See Figure 23). These views are afterwards dynamically added to the layout, using the `addView()` method. This design choice avoids hardcoding UI elements in XML for each character and allows flexible rendering based on character data, improving modularity and scalability.

4.3.1.2 Dynamic view creation and layout

Instead of creating an XML file for each character, we chose to make a reusable method, `createCharacterView()`. This constructs each character's visual representation. The function uses `LinearLayout` containing an image and a text label, both of which are found in the character information (See Figure 25 & 26). The layout is styled dynamically based on the characters theme colour. `Layout`, `ImageView` and `TextView` objects are used for implementing the data of the characters to the layout.

```
48      LinearLayout layout = new LinearLayout(context: this);
49      layout.setOrientation(LinearLayout.HORIZONTAL);
```

Figure 24 `createCharacterView()` `LinearLayout` setup

Layout handles the orientation of the image and name, making it horizontal and sets the layout to be interactive, as seen in seen in Figure 24 above.

```
69     ImageView imageView = new ImageView( context: this);
70     imageView.setImageResource(character.getImageResourceId());
71     LinearLayout.LayoutParams imageParams = new LinearLayout.LayoutParams( width: 300, height: 300);
```

Figure 25 createCharacterView() ImageView

```
76     TextView nameView = new TextView( context: this);
77     nameView.setText(character.getCategory());
```

Figure 26 createCharacterView() TextView

This implementation ensures that the UI components are generated consistently while making it easy to maintain the logic.

4.3.1.3 User Interaction and Navigation

The character views are made interactive with the `OnClickListener` object. This allows the user to click on a character and launch the `CharacterDetailActivity`, implemented as seen in Figure 27 below.

```
90     layout.setClickable(true);
91     layout.setFocusable(true);
92     layout.setOnClickListener( View v -> {
93         Intent intent = new Intent( packageContext: this, CharacterDetailActivity.class);
94         intent.putExtra( name: "name", character.getName());
95         startActivity(intent);
96     });
```

Figure 27Character click intent setup

Using `Intent.putExtra()` allows data to be passed between activities. The next example shows how the name value is used to remember the user's selected character.

4.3.2 Showing the character's description

After navigating to the activity that displays the description of the selected category, the application needs to identify which character that were selected. This activity has access to the list of characters, and the name determines the selected character and is transferred via the `getStringExtra()` method (See Figure 28). A `for` loop is then used to cycle through the list until a match is found. This approach assumes that two characters do not have the same name. In a fully implemented prototype, each of the characters would either need to use the name as the primary key in the database or be assigned a separate number ID for identification. The `if` statement checks whether the name is `null`, if `TRUE`, it logs an error using `Log.e()`.

This method prints an error message to Logcat, which is useful for debugging. In this case, it helps developers identify that no name was passed through the `Intent` and if the name is present, the loop will begin.

```
53     Intent intent = getIntent();
54     String name = intent.getStringExtra("name");
55
56     if (name == null) {
57         Log.e("CharacterDetailActivity", "Character name not found in intent");
58         return;
59     }
60
61     for (int i = 0; i < charactersList.size(); i++) {
62         if (charactersList.get(i).getName().equals(name)) {
63             currentIndex = i;
64             break;
65         }
66     }
```

Figure 28 Retrieving the Selected Character from Intent and Matching by Name

On this page, we made it possible to cycle through the descriptions of each character with the code below that changes the selected character and updates the view when the left or right button is clicked (see Figure 18, 3.B). Lambdas are used in place of the *single use methods*, or *functional interfaces*, which create a new `OnClickListener` object and defines what happens when the button is pressed. The parameter `v`, on the left of the arrow, is a `View` object. We have skipped explicitly writing the initialisation of the `OnClickListener` object. In curly brackets on the right of the arrow is the functionality of the `onClick()` method of the `OnClickListener` object, which defines what should happen when the button is clicked.

The left arrow and the right arrow, called `btnNext` and `btnPrevious` respectively, work similarly to each other. When clicked, 1 is subtracted or added to the index over the character list, as seen in Figure 29 below. The modulus operators are used as a way to loop the list. If the user is currently at the first character and presses the back button, the product of the modulus operator will be 4, putting the user at the other end of the list.

```

72     btnBack.setOnClickListener( View v -> finish());
73
74     btnPrevious.setOnClickListener( View v -> {
75         currentIndex = (currentIndex - 1 + charactersList.size()) % charactersList.size();
76         updateCharacterView();
77     });
78
79     btnNext.setOnClickListener( View v -> {
80         currentIndex = (currentIndex + 1) % charactersList.size();
81         updateCharacterView();
82     });

```

Figure 29 Character Navigation Using Left and Right Arrows

4.3.3 Map

Initially, we intended to implement Google Maps as a component of the application. In situations where new students needed to navigate the city, the map would assist them in travelling from point A to point B and, as a latent effect, help them become more familiar with their surroundings. It would support students who might get lost in the city and serve as an organic way to display various events and venues. We planned to use the coordinates and addresses of different venues and events to link them to specific locations associated with the characters' sites. However, Google Maps will not be included in this prototype, as our current knowledge of integrating Google Maps into the application is insufficient, even though we did research the possibility of integrating Google Maps. Nevertheless, the mapping feature remains an important component to consider for implementation in future iterations.

4.3.4 Timeline Of Events

The `TimelineOfEventsActivity` class is responsible for displaying a timeline interface that contains two events related to the selected character. The screen is dynamically updated to show the chosen character's details, with implementations of clickable UI components like category tabs, navigation arrows and action buttons to access the calendar and home shortcuts (see Figure 18, 3.E).

After selecting a character, the user is moved to the timeline of the selected character. When `TimelineOfEventsActivity` is activated, the `onCreate()` method initialises the visual layout by using `activity_timeline_of_events`. This XML file combines the `LinearLayout`, `FrameLayout`, and a `CoordinatorLayout` to structure and position the components: The timeline image, category tabs, event bubbles, and the navigation arrows.

After this, key visual components such as the character's name, image, timeline, and background are connected using `findViewById()` (see Figure 30).

```
35      characterNameTextView = findViewById(R.id.characterNameTextView);
36      characterImageView = findViewById(R.id.characterImageView);
37      timelineImage = findViewById(R.id.timelineImage);
38      backgroundContainer = findViewById(R.id.backgroundContainer);
```

Figure 30 Connecting UI Components Using `findViewById()`

4.3.5 Event Bubbles and Timeline Display

The layout includes two pre-designed event bubbles, which are added via an `<include>` tag in the XML file. The event bubbles display information about upcoming events like date, title, and a clickable “see event” button, which makes it possible for the user to read more about the event (see Figure 31).

```
40      View eventBubble1 = findViewById(R.id.eventBubble1);
41      TextView event1Date = eventBubble1.findViewById(R.id.eventDate);
42      TextView event1Title = eventBubble1.findViewById(R.id.eventTitle);
43      Button event1Button = eventBubble1.findViewById(R.id.eventButton);
44      View event1Divider = eventBubble1.findViewById(R.id.eventDivider);
```

Figure 31 Accessing Elements Inside an Included Event Bubble

This setup is mirrored for the second event bubble, making it possible for each character to display two event bubbles in each of their timelines.

4.3.5.1 Character Matching

For the activity to know what character to display, it gets the character's name from the previous activity using an `Intent`. The name of the selected character is retrieved in line 59, as seen in Figure 32 below.

```
59      String characterName = getIntent().getStringExtra( name: "characterName");
```

Figure 32 Retrieving the Selected Character Name via `Intent`

Afterwards, a loop goes through the list of characters to find the one that matches the name, as seen in Figure 33 below.

```

60         for (int i = 0; i < charactersList.size(); i++) {
61             if (charactersList.get(i).getName().equals(characterName)) {
62                 currentIndex = i;
63                 break;
64             }
65         }

```

Figure 33 Finding the Selected Character in the List

When the selected character is found, its index is stored, and the timeline can be updated to match the selected character. The `equals()` method is used instead of the equality (`==`) operator, because that would compare the strings' pointers rather than their contents. An alternative solution for retrieving the character object would be in the user's profile as a variable and accessing the character from there.

4.3.5.2 Updating the Screen with Character Data

The `updateTimeline()` method is called to dynamically update the activity based on the selected character. When the method is called, it retrieves the `Characters` object using the current index on line 127, in Figure 34 below.

```

127         Characters c = charactersList.get(currentIndex);

```

Figure 34 Accessing the Selected Character Object by Index

Afterwards, the character's theme colour is accessed, and a lighter version of the character's theme colour is created using the method in line 129 and 130, as seen in Figure 35 below.

```

129         int themeColor = c.getThemeColour();
130         int lightBackgroundColor = Colours.lightenColor(themeColor, factor: 0.7f);

```

Figure 35 Retrieving and Adjusting the Character's Theme Colour

The lighter version of the theme colour is applied to the background using a `GradientDrawable` object to make a personalised design for each of the characters (see Figure 36).

```

133         View roundedBackground = findViewById(R.id.roundedBackground);
134         GradientDrawable backgroundDrawable = (GradientDrawable) roundedBackground.getBackground();
135         backgroundDrawable.setColor(lightBackgroundColor);

```

Figure 36 Styling the Background with a GradientDrawable

4.3.5.3 Event Matching and Display

Each character has two events displayed in the activity. These are identified through a loop which filters the list of events in line 142 – 146 in Figure 37 below. The `for` each loop counts up the `count` variable using the post increment operator (`++`), meaning that `count` is incremented, after the event has been assigned to the given position in `matchedEvents[]`.

```
142     Event[] matchedEvents = new Event[2];
143     int count = 0;
144     for (Event event : eventList) {
145         if (event.getCharacterName().equalsIgnoreCase(c.getName()) && count < 2) {
146             matchedEvents[count++] = event;
147         }
148     }
149
150     if (matchedEvents[0] != null) {
151         date1.setText(matchedEvents[0].getDate());
152         title1.setText(matchedEvents[0].getTitle());
153         div1.setBackgroundColor(themeColor);
154         btn1.setBackgroundTintList(ColorStateList.valueOf(themeColor));
155         btn1.setTextColor(Color.WHITE);
156         btn1.setOnClickListener( View v -> navigateToEvent(matchedEvents[0].getVenue().getName()));
157     } else {
158         date1.setText("");
159         title1.setText("");
160         div1.setBackgroundColor(Color.TRANSPARENT);
161         btn1.setBackgroundTintList(ColorStateList.valueOf(Color.TRANSPARENT));
162         btn1.setOnClickListener(null);
163     }
```

Figure 37 Matching and Displaying Events for the Selected Character

Then the matching events are inserted into the bubble views in line 151 - 156 in Figure 37 above, and each event has a title and date displayed as well as a clickable “See event” button, which gives the users the possibility to read more about the specific event. If there are fewer events than two, the event bubbles are cleared and made unclickable. The same code, from line 150 – 163 above, is used for the second event window as well.

4.3.5.4 Navigating Between Characters

To let the user switch between character timelines without having to go back to the selection page, two arrow buttons `btnTimelineNext` and `btnTimelinePrev` are provided on each side of the screen.

```

71     btnNext.setOnClickListener( View v -> {
72         currentIndex = (currentIndex + 1) % charactersList.size();
73         updateTimeline(characterNameTextView, characterImageView, timelineImage, backgroundContainer,
74             event1Date, event1Title, event1Button, event1Divider,
75             event2Date, event2Title, event2Button, event2Divider);
76     });
77
78     btnPrev.setOnClickListener( View v -> {
79         currentIndex = (currentIndex - 1 + charactersList.size()) % charactersList.size();
80         updateTimeline(characterNameTextView, characterImageView, timelineImage, backgroundContainer,
81             event1Date, event1Title, event1Button, event1Divider,
82             event2Date, event2Title, event2Button, event2Divider);
83     });

```

Figure 38 Navigating Between Characters Using Arrow Buttons38

When the buttons are pressed, they increase or decrease the `currentIndex` and then change the interface using `updateTimeline()`. All the interface views and objects that change are passed as parameters into the function which updates them using the functionality in Figure 38 above.

4.3.5.5 Category Tabs

At the top of the screen, there are five options of categories: Social, Music, Creative, Sporty and Geeky (see Figure 18, 3.E & 3.G). These are displayed as `TextView` elements and are clickable, which allows users to switch directly to other characters based on the users' interests. Each of the tabs uses a shared listener, which can be seen in Figure 39 below.

```

91     View.OnClickListener categoryClickListener = View v -> {
92         String clickedCategory = ((TextView) v).getText().toString();
93         for (int i = 0; i < charactersList.size(); i++) {
94             if (charactersList.get(i).getCategory().equalsIgnoreCase(clickedCategory)) {
95                 currentIndex = i;
96                 updateTimeline(characterNameTextView, characterImageView, timelineImage, backgroundContainer,
97                     event1Date, event1Title, event1Button, event1Divider,
98                     event2Date, event2Title, event2Button, event2Divider);
99                 break;

```

Figure 39 Handling Category Tab Clicks with a Shared Listener

When one of the category tabs is clicked, they trigger a search in the character list for the first match in that category. This updates the screen using the `updateTimeline()`. The listener is implemented using a lambda expression to simplify the code. Instead of creating a separate `OnClickListener` class, the lambda directly defines the behaviour of the click event. In this case, the clicked `TextView` is converted into a string to get the selected category, whereas a loop checks each character to find the first one with a matching category. When found, the interface is updated accordingly.

4.3.5.6 Home and Calendar Icons

At the bottom of the screen, there are two clickable icons which provide the users with shortcuts to get faster access to other parts of the app. The calendar icon opens `DateActivity` and displays all events in a date-based format (see Figure 40).

```
111     ImageView calendarIcon = findViewById(R.id.calendarIcon);
112     calendarIcon.setOnClickListener( View v -> {
113         Intent intent = new Intent(
114             packageContext: TimelineOfEventsActivity.this, DateActivity.class);
115         startActivity(intent);
116     });
117
118     ImageView homeIcon = findViewById(R.id.homeIcon);
119     homeIcon.setOnClickListener( View v -> {
120         Intent intent = new Intent(
121             packageContext: TimelineOfEventsActivity.this, TimelineOfEventsActivity.class);
122         startActivity(intent);
123     });
```

Figure 40 Shortcut Icons for Calendar and Home Navigation

Whereas the home icon reloads the page by creating an intent which transfers the user to the same activity, this sets the view elements to display the information associated with the first chosen character. The implementation of these small details improves usability by offering the users multiple ways to navigate through the app (see Figure 18, 3.E & 3.G).

4.3.5.7 Viewing the Full Event Details

The two event views displayed in the `TimelineOfEventsActivity` includes a “See Event” Button, which allows the user to learn more about the specific event. When the button is clicked, the user is transferred to the activity `EventDetailsActivity` where all the information on the event is displayed. This includes a description of the event, image of the selected character, the venue name, and the date of the event (see Figure 18, 3.F).

```

201         if (matchedEvents[1] != null) {
202             date2.setText(matchedEvents[1].getDate());
203             title2.setText(matchedEvents[1].getTitle());
204             div2.setBackgroundColor(themeColor);
205             btn2.setBackgroundTintList(ColorStateList.valueOf(themeColor));
206             btn2.setTextColor(Color.WHITE);
207             btn2.setOnClickListener(View v -> navigateToEvent(matchedEvents[1].getTitle()));
208         } else {
209             date2.setText("");
210             title2.setText("");
211             div2.setBackgroundColor(Color.TRANSPARENT);
212             btn2.setBackgroundTintList(ColorStateList.valueOf(Color.TRANSPARENT));
213             btn2.setOnClickListener(null);
214         }
215     }
216     /**
217      * Opens the event details screen for a given venue name.
218      * @param eventTitle the name of the event
219      */
220     2 usages
221     public void navigateToEvent(String eventTitle) {
222         Intent intent = new Intent(packageContext, this, EventDetailsActivity.class);
223         intent.putExtra("name", eventTitle);
224         startActivity(intent);
225     }
226 }

```

Figure 41 Clickable button and Intent for Navigating to Event Details

Inside the `updateTimeline()` method in `TimelineOfEventsActivity`, a listener object is set on the event buttons in line 207. The button calls the `navigateToEvent(String venueName)` method, in line 220 which creates an `Intent` to open the `EventDetailsActivity`. The event title is passed as an extra piece of data in line 222, as seen in Figure 41 above. This approach is used because event titles are used as identifiers to locate the event in the next activity.

4.3.5.8 Receiving and Displaying Event Details

In `EventDetailsActivity`, the `onCreate()` method retrieves the venue name that was passed from `TimelineOfEventsActivity` in line 56 - 57.

```

56     Intent intent = getIntent();
57     String venueName = intent.getStringExtra( name: "name");
58
59     for (int i = 0; i < eventList.size(); i++) {
60         if (eventList.get(i).getVenueName().equalsIgnoreCase(venueName)) {
61             currentIndex = i;
62             break;
63         }
64     }
65
66     Event currentEvent = eventList.get(currentIndex);
67
68     // Set data into views
69     titleView.setText(currentEvent.getTitle());
70     venueView.setText(currentEvent.getVenueName());
71     dateView.setText(currentEvent.getDate());
72     descriptionView.setText(currentEvent.getDescription()); // Long description displayed here
73     pictureView.setImageResource(currentEvent.getImageResourceId());

```

Figure 42 Retrieving and Displaying Event Information in EventDetailsActivity

The correct event is found similarly to how the correct character is found in Figure 42 above. When the accurate `Event` object is found, the event's information is put into the layout using the `setText()` and `setImageResource()` methods, as seen in line 69 - 73. This fills the screen with the full details of the selected event.

4.3.5.9 Exit Button

An exit button is provided in the top-left corner of the `EventDetailsActivity`, this allows users to return to the timeline view in line 48 in Figure 43 below. Exit buttons in the app work the same way, using the `finish()` method. That method takes the user back to the last activity they came from.

```

47     // Handle back button click
48     btnBack.setOnClickListener( View v -> finish());

```

Figure 43 Exit Button Implementation Using finish()

4.3.6 Making the calendar

Another section of the application development is the calendar (see Figure 18, 3.G). Starting the calendar activity, after assigning the view objects and their corresponding XML tags, these three methods set up the screen, see Figure 44 below.

```

73      // Set initial theme and data
74      applyCharacterTheme(selectedCharacter);
75      loadCharacterEvents();
76      updateCalendar();

```

Figure 44 Setting Initial Theme and Loading Events in onCreate()

applyCharacterTheme() uses the fake database to get the correct character's theme colour and assign that colour to the background object, using a for each loop similar to the function loadCharacterEvents(), see Figure 45 below. This method updates the Hash Map object, called eventMap, we use to store the calendar's dates and events. Hash Map objects behave very similarly to Hash Tables (*HashMap (Java Platform SE 8)*, n.d.).

```

153     private void loadCharacterEvents() {
154         eventMap.clear();
155         FakeDatabase fakeDb = new FakeDatabase();
156         for (Event event : fakeDb.getEventList()) {
157             if (event.getCharacterName().equalsIgnoreCase(selectedCharacter)) {
158                 String dateKey = convertToDateKey(event.getDate());
159                 if (dateKey != null) {
160                     eventMap.put(dateKey, event);
161                 }
162             }
163         }
164     }

```

Figure 45 Matching Character Events with loadCharacterEvents()

We utilise the convertToDateKey() method in line 158, to format the events given date to a string containing the year, month, and day, connected by dashes. The events' date is by default saved as a string formatted like "Tuesday 03 - 06" and the eventMap table uses the year-month-day format, so a conversion is needed.

```

241  @      private String convertToDateKey(String eventDate) {
242          String[] parts = eventDate.split(regex: " ");
243          if (parts.length == 2) {
244              String[] dateParts = parts[1].split(regex: "-");
245              int day = Integer.parseInt(dateParts[0]);
246              int month = Integer.parseInt(dateParts[1]);
247              int year = currentCalendar.get(Calendar.YEAR);
248              return String.format("%04d-%02d-%02d", year, month, day);
249          }
250          return null;
251      }

```

Figure 46 Converting Event Dates with convertToDateKey()

The method, shown in Figure 46 above, uses the `split()` method to separate the `eventDate` string into parts. Each part is set to be separated by a space and then put into the `parts[]` array (line 244). Assuming we, for example, have a “Tuesday” and a “03 - 06” part, we split up the latter part so each separately can be put into the new string (line 245 - 246). The `currentCalendar` is a `Calendar` object that was initialised at the beginning of this activity, we use it to get the current year on line 248.

4.3.6.1 Updating the calendar

Now the `dateKey` string has been returned into the `eventMap` table (line 158, Figure 45), the `updateCalendar` method is called, as seen in Figure 47 below. Like with the other “update” methods in our app, this one is used to update the interface when the current character is changed. It starts by clearing the list of integers `daysWithEvents` (line 169), storing what days in the current month the relevant character has events on. Because the `eventMap` object (line 173) now contains the events and their dates, we can use this `for each` loop to add all events that are taking place in the currently selected month. This works because the key to each entry is the event’s date, we can *split* it, only getting the day the same way we *split* the `eventDate` string from Figure 45.

```

168     private void updateCalendar() {
169         daysWithEvents.clear();
170
171         String currentMonthKey = getMonthKey(currentCalendar);
172
173         for (Map.Entry<String, Event> entry : eventMap.entrySet()) {
174             String key = entry.getKey();
175             if (key.startsWith(currentMonthKey)) {
176                 String[] parts = key.split(regex: "-");
177                 int day = Integer.parseInt(parts[2]);
178                 daysWithEvents.add(day);
179             }
180         }

```

Figure 47 Identifying Calendar Days with Events in updateCalendar()

Knowing what days, the current month has events (line 160 from Figure 45) is used in the next part of the method, where it is used when creating all the dates we put into the calendar.

For the purpose of organising dates, we made a `DateModel` class that contains the date as a one- or two-digit integer, a Boolean for tracking if the date belongs to the selected month, and a Boolean for tracking whether it has an event or not.

```

182     List<DateModel> days = new ArrayList<>();
183     Calendar temp = (Calendar) currentCalendar.clone();
184     temp.set(Calendar.DAY_OF_MONTH, 1);
185     int paddingDays = temp.get(Calendar.DAY_OF_WEEK) - 1;
186     int daysInMonth = temp.getActualMaximum(Calendar.DAY_OF_MONTH);
187
188     for (int i = 0; i < paddingDays; i++) {
189         days.add(new DateModel( day: 0, isCurrentMonth: false));
190     }
191
192     for (int i = 1; i <= daysInMonth; i++) {
193         boolean hasEvent = daysWithEvents.contains(i);
194         days.add(new DateModel(i, isCurrentMonth: true, hasEvent));
195     }
196
197     SimpleDateFormat sdf = new SimpleDateFormat( pattern: "MMMM yyyy", Locale.getDefault());
198     monthText.setText(sdf.format(currentCalendar.getTime()));
199
200     adapter = new CalendarAdapter(days, onItemClickListener: this, characterThemeColor);
201     calendarRecyclerView.setAdapter(adapter);
202 }

```

Figure 48 Populating Calendar Grid and Assigning Adapter in updateCalendar()

Lines 184 - 186 in Figure 48 above, use a copy of our calendar object, called `temp`. The `clone()` method returns a calendar of type `Object` and therefore it is necessary to manually

type cast it to a `Calendar` type, using `(Calendar)` from line 183. The integer `paddingDays` is used to create the blank spaces in our calendar. They are the days in the displayed calendar grid that do not belong to the current month, and those are blank and greyed out. The `daysInMonth` integer contains the number of days in the current month. Together they always add up to 35 so that our calendar can be 7 by 5 grid. Each `DateModel` objects is created in the for loops on lines 188 - 195 and the relevant dates will have their `hasEvent` boolean set to true. The second to last part of the `updateCalendar()` method (lines 197 - 198) creates a `SimpleDateFormat` object to use its `format()` method in order to get the text displayed in the `monthText` view. The last part of the method assigns the adapter object a new instance of our `CalendarAdapter` class (lines 200 - 201). It uses our list of `DateModel` objects in order to create and handle the display of the calendar and make each date with an event interactable.

4.3.7 Using RecyclerView

Android Studio's `RecyclerView` is a `ViewGroup` element that we use to contain all the calendar dates we wanted to display, and it helps our application save memory and makes the calendar activity more responsive by recycling the views used to hold each date, rather than deleting and creating new ones each time (*Android Developers*, n.d.). The views that `RecyclerView` recycles are `ViewHolder` objects, so in our application each `ViewHolder` corresponds to one date in the calendar. Therefore, each holds a `TextView` and an `OnItemClickListener` object, the latter being responsible for listening for when the calendar date is clicked, see Figure 49 below.

```
96      public static class ViewHolder extends RecyclerView.ViewHolder implements View.OnClickListener {  
97          TextView dateText;  
98          OnItemClickListener onItemClickListener;
```

Figure 49 `ViewHolder` Class with `RecyclerView`

When a date is clicked, we use a `try catch` statement to make sure we only handle strings that can be converted to integers, as the alternative would be a thrown `NumberFormatException` exception (lines 119-123, Figure 50). A date of 0 means that one of the empty grids in the calendar was clicked, as shown in Figure 48, line 189, and only actual dates should have the possibility of responding to a click.

```

116      @Override
117      public void onClick(View v) {
118          int day = 0;
119          try {
120              day = Integer.parseInt(dateText.getText().toString());
121          } catch (NumberFormatException e) {
122              // ignore invalid clicks on empty dates
123          }
124          if(day != 0) {
125              onItemClickListener.onItemClick(day);
126          }
127      }

```

Figure 50 Handling Click Events on Calendar Dates Using ViewHolder

The `onItemClick()` method, called on line 125, exists in our Calendar Activity class, and it updates the event text and picture displayed to the newest selected event. It also makes the view for the event visible or invisible, depending on whether the clicked date has an event. If it does not, the view is made invisible.

4.3.7.1 ViewHolder creation

The RecyclerView element calls the methods to create and populate each ViewHolder when needed (*Android Developers*, n.d.). The first methods are called `onCreateViewHolder()` and creates and returns an empty view object and `onBindViewHolder()` (see Figure 51) which defines the look of each of the views. The `onBindViewHolder()` method works by using the `DateModel` object for the given date to know how to colour the background and the number that the given date has.

```

61      @Override
62      public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
63          DateModel model = dateList.get(position);
64
65          if (model.day == 0) {
66              holder.dateText.setText("");
67              holder.dateText.setAlpha(0f);
68              holder.itemView.setClickable(false);
69          } else {
70              holder.dateText.setText(String.valueOf(model.day));
71              holder.dateText.setAlpha(model.isCurrentMonth ? 1.0f : 0.3f);
72
73              // Highlight event days (optional)
74              if(model.hasEvent) {
75                  holder.dateText.setTextColor(eventColor);
76              } else {
77                  holder.dateText.setTextColor(holder.itemView.getResources()
78                      .getColor(android.R.color.black));
79              }
80
81              holder.itemView.setClickable(true);
82          }
83      }

```

Figure 51 Populating Calendar Dates in `onBindViewHolder()`

The `position` parameter is the given `ViewHolder`'s position in the list of views that `RecyclerView` creates, and it corresponds to the position the given date should have in the calendar. We utilize this on line 63. The if else statement from line 65 makes the `ViewHolder` a gray and unclickable box without text if it is not a part of the current month, as indicated by the day being equal to zero. The else condition on line 69 onward first writes the day number, then on line 71 uses a ternary operator when deciding the number opacity. If `isCurrentMonth` is true, then the opacity is set to 1.0f or 100 percent. Otherwise, the opacity is 30 percent. Then, from line 74, the else condition makes the colour of the text the same colour as the character's theme colour if the date contains an event. It otherwise defaults to black and lastly makes the view clickable.

5 Evaluation of Prototype

In the following section, we evaluate our prototype. The evaluation is based on Jakob Nielsen and Rolf Molich's ten usability heuristics. This evaluation is conducted by us as designers, drawing on our domain-specific knowledge and relevant design experience. Our final prototype of the application has been continuously evaluated, these evaluations are organised according to the ten heuristics (Zelezny, 2024).

1. Visibility of system status

The application is designed to maintain user awareness by consistently keeping them informed on their current position within the system. A still image of a map is included on both the login and event pages, to provide immediate context cues about the applications purpose, and the physical locations of events. This helps ground the user experience and reinforces the event-based focus. The navigation bar highlights the current selected page, making it easier for users to stay oriented and understand where they are within the applications structure. These features offer visual feedback that supports user awareness (Zelezny, 2024).

2. Match between system and the real world

The applications design reflects real-world conventions familiar to the target audience. Event locations are displayed on maps, providing concrete spatial references. The timeline interface draws on progression metaphors found in well-known mobile games like Duolingo and Candy Crush, creating a sense of familiarity. Navigation icons and buttons follow industry standards, and the characters act as mascots, adding personality to the categories without introducing complexity. The language used throughout the application is simple and conversational. These choices help bridge the gap between digital interaction and real-world expectations (Zelezny, 2024).

3. User control and freedom

The application allows users to navigate while maintaining a structured experience. An example of this is allowing users to move back and forth between screens, undo selections such as chosen characters, and exit pages easily using a consistent exit button in the top-right corner.

Multiple navigation methods, like swiping, pressable arrows, and the header displaying categories, give users multiple alternative ways to interact with the app. While the overall flow is guided to reduce the likelihood of mistakes, while still giving users the freedom to explore and correct actions if they make ‘slips’ (Norman, 2013). However, the application is relatively rigid, offering limited options, which also helps reduce user error. The characters are based on stereotypes, which may restrict user freedom, if the user find it difficult to connect with their attributes.

4. Consistency and standard

Visual and functional consistency is a core part of the entire design. Navigation icons, button behaviour, colour schemes, and layouts are uniform across all pages of the application, both in appearance and functionality. This ensures consistency by reducing the likelihood of possible confusion for the user navigating the application. Obvious placed exits (X), in the top right corner, to close pages are used to meet user expectations, assuming said users are familiar with platforms that follow industry standards. These design choices help minimise confusion and make the interface feel familiar, hopefully giving the user an intuitive experience.

5. Error prevention

The application minimises the potential for user errors by providing clear affordances and reversible actions. For example, character selections are not permanent, and can be changed at any time, The consistent exit buttons make it easy to leave unwanted pages. The structured navigation helps prevent users from making slips, while the design anticipates common slips and offers easy ways to undo actions.

6. Recognition rather than recall

Colour-coding is used throughout the application to visually connect characters, categories, and events, making it easy for users to recognise patterns and relationships. Consistent page layouts and clearly labelled categories reduce cognitive effort and support intuitive use, making it clear to the users that every category contains familiar functionality. Users do not need to memorise what each character represents, as the visual cues, descriptions and structure reinforce the relevant associations across the interface.

7. Flexibility and Efficient Use

The application supports both new and experienced users through multiple interaction methods. Swiping, arrow buttons, and tap-based selections all allow for fluid navigation. Users can quickly jump between characters, categories, and events based on their preferences, while still following a guided structure that avoids confusion. This balance between flexibility and simplicity makes the app accessible and efficient for a wide range of users.

8. Aesthetic and minimalist design

The interface presents information in a clean and simple way. Events are shown as clickable points on a timeline, and the event details are kept brief. While the colour scheme and character design necessarily not be considered minimalist, the use of colours and character illustrations adds visual interest without introducing clutter. The design of the characters is neutral and avoids gender connotations, aiming to prevent users from being biased in their selection. Although the aesthetic leans toward a playful style, it remains purposeful and organised, balancing personality with usability. The overall visual design supports clarity and intention without being overwhelming.

9. Help Users Recognize, Diagnose, and Recover from Errors

While the application does not include a tutorial, it offers intuitive navigation tools that make it hard for the user to make errors that are difficult to recover from. The only implemented error message in the application is an “Invalid Password”, which appears if users enter any password other than the predefined default password. The wording of the error message is clear and understandable to help users identify what error has occurred. Apart from this, the application does not output error messages, as there are no other options for users to provide input.

10. Help and Documentation

We acknowledge the importance of onboarding support to help users familiarise themselves with the application and make effective use of its features. However, the function was not implemented in the final version of the application. Instead, the design changes aimed to reduce the need for such help by clarifying navigation and interface logic. Based on our evaluation using Nielsen and Molich’s ten usability heuristics, we claim that the final application offers a coherent and user-friendly experience. The prototype was designed with attention to

consistency, recognisability, and error prevention, which we believe supports user orientation and navigation. Although the application lacks certain features like onboarding and broader input feedback, we argue that the overall structure and design choices help mitigate these gaps. In the following discussion, we reflect on these claims, critically examine our design decisions, and consider how the application could be improved through further development and user testing.

6 Discussion

In the following discussion and evaluation of the project, we aim to reflect on our various decisions and limitations, while also considering what could have been done differently. Throughout the development process, the project underwent a significant shift in focus. Initially, the project focused directly on the relationship between two students and how to facilitate the formation of such connections. However, human relationships are not only conceptually diverse but also inherently complex (2.1.1).

As a result, we choose to redirect our focus toward events, aiming to make it easier for individuals to find gatherings aligned with their interests, thereby creating opportunities for social interaction. This conceptual shift made it possible for the participants in our workshop to engage with the application in a way that differed from their initial expectations, as one participant said:

“I thought the application would relate more to meeting new people as we talked about in the workshop” (Appendix 2).

While they anticipated a solution centred around one-on-one connection-building, the revised prototype was instead oriented toward helping users discover relevant events. Some aspects of the workshop touched on event discovery; its primary focus was originally on helping students build a network as newcomers to the city. This led to a misalignment between the workshop/interview content and the prototype that was tested. This mismatch between workshop expectations and the tested prototype created some friction in feedback relevance.

Although the underlying intent of using events as a means for social integration remained, it became less central as the project progressed. Overall, the workshop and integration of interviews did not primarily unfold as expected. Nevertheless, we gathered valuable data, and the group setting facilitated critical discussions about existing platforms. However, the workshop format lacked structure and evolved into more of a group discussion between the facilitators and participants, with ideas being noted on Post-it notes. This posed difficulties in leveraging the participants' creativity, as they did not have anything to rely on, other than existing websites and apps. An example of one of the ideas that emerged from the workshop was a concept similar to that of Tinder, but for events and friends and not dating. One benefit

of this approach was the feedback regarding the critique of existing apps and websites like Facebook, which guided the intention of how the application would present and save events. This was evident in quotes like:

“..About how difficult it can be to find the event you're looking for the 'Relevant for you this week' section on FB doesn't show either the ones you're interested in or the ones you're searching for.” (Appendix 1).

In retrospect, a more effective approach might have been to conduct interviews prior to the workshop with time to analyse their responses; this could have allowed us to gather deeper insights into the target population and problem domain. Subsequently, a workshop could have been organised where participants were tasked with collaboratively building solutions. This could have fostered more exchanges among participants and allowed them to express their ideas more freely through co-creation. Furthermore, such an approach could have addressed a key challenge in participatory design. Participants' limited awareness of design possibilities. By providing "building blocks", we could have encouraged creativity and reduced the constraints imposed by participants' preconceptions. This strategy may also have facilitated a more complete implementation of the three phases of the Future Workshop.

6.1 Usability Testing

In designing our usability test, we based our interview questions on Nielsen's Ten Usability Heuristics. While these heuristics are commonly used for evaluating user interfaces, we have since realised that they are not intended for informing user interview questions during early or exploratory testing. Through this project, we've learned that Don Norman's Seven Principles for Transforming Difficult Tasks into Simple Ones (Norman, 2013, pp. 175–187) would have been a more appropriate framework, as they focus on underlying cognitive challenges, task simplification, and designing for intuitive understanding. These principles could have guided our test design toward a better understanding of how users make sense of the interface, rather than just spotting the surface-level usability issues, as we experienced in our case. This insight has strengthened our understanding of when and how to apply different UX frameworks, and in future projects, we will be more deliberate in selecting the framework that best aligns with the purpose of our evaluation.

We conducted the usability tests on our initial prototype, which led to the development of a second version. Our usability test had a total of four testers, who individually explored the

high-fidelity prototype. One common issue with usability testing of prototypes is that feedback often focuses disproportionately on visual design rather than functional aspects. Although the prototype incorporated several interactive elements and user flows, most of the changes made between the first and second versions were cosmetic rather than functional, hence the surface-level usability issues.

Despite this, participants provided insightful comments, and by observing their task completion processes, we were able to infer certain functional shortcomings this gave opportunities for improvement. These observations helped guide us toward aligning the app more closely with industry standards. Ideally, we would have conducted usability testing on a fully coded version of the application, instead of a high-fidelity prototype made in Figma. This would have enabled us to design specific tasks for participants and collect more actionable data regarding the application's functionality, rather than its interface alone. It would have allowed us to test all of Nielsen's heuristics and not have to skip testing the applications ability to handle error messages and helping the user recover from them.

Although some feedback addressed functionality, it was primarily in terms of features participants wished the application had, rather than evaluations of how existing features performed, for example, "The background should become the map, and the close X-button shouldn't be at the bottom of the page." (Appendix 2) This problem may also be due to the participants' lack of knowledge on what is functionally possible, this can generally be considered a problem in participatory design. We tried to accommodate this by formulating questions and tasks on the 10 heuristic principles. We came to understand that Nielsen's heuristics are not originally intended to be formulated as interview questions but rather serve as expert evaluation criteria for assessing the usability of prototypes. This made us realise we needed to conduct an analysis based on our data collection using the severity rating (section 3.2.3.4). Since Nielsen's 10 heuristics are not designed as a method for data analysis or for generating interview questions, they ultimately served as a source of inspiration for our evaluation framework rather than a method. The limitations observed could stem from the heuristics' reduced applicability to modern students, whose technological needs and abilities differ from those prevalent when these heuristics were initially developed. The traditional heuristic evaluations may not fully capture the evolving expectations and behaviours of modern users, especially when considering advancements in technology and changes in user interaction patterns, and the purpose of the heuristics is not to create interview questions or operationalise

“functionality”. Due to the users not being able to do the test on a fully functional version of the application, we recognise that we could not conclusively determine whether the final product would meet the expectations and requirements of the intended user group.

Finally, the evaluation of usability data was constrained by the use of Nielsen’s 10 heuristics. While these principles served as valuable guidelines for framing our observations and questions, for example, in the interview question, “To what extent did the buttons behave like you expected? Did they bring you to the places you expected?” (Appendix 2). This question came from the inspiration of the heuristic principle “Consistency and standards” and “User control and Freedom” (section 2.4.3). They also presented challenges in translating user responses into specific, system improvements, as many of the answers remained abstract rather than directly tied to interface functionality for example: “The buttons make sense” (Appendix 2), which is good but also does not give us as designers a lot of opportunity to make sufficient changes. Instead, we ended up applying the severity rating. Ultimately, we prioritised and ranked user feedback based on severity and feasibility of implementation within the application. The usability tests were therefore particularly useful in identifying which aspects of the prototype captured the participants' attention and prompted suggestions for improvements that could enhance user engagement.

In the process of system development, the choices made by the group became evident. As a group, we reflected on what aspects should be prioritised during system development specifically. Whether the diagrams should include all components, even those we do not intend to implement in the final product due to skill constraints, or only the components we plan to realise. We decided that the diagrams should illustrate the full extent to which the application could be developed to accommodate user needs (section 1.6), as a way of demonstrating a complete implementation framework see section.

The most critical components were selected by ranking the requirements alongside user statements gathered during the usability test. This approach enabled us to develop a product that addressed the most important elements identified from the data, resulting in a functional application, albeit one that lacks certain features such as calendar event integration or GPS tracking. The requirements that were not prioritised or implemented will be discussed in the 'Future Work' section of the project, including an analysis of how their inclusion could have impacted the application.

7 Future Work

Although the current version of the application meets the initial design goals, several areas could be further refined and developed in future iterations. One potential improvement is the addition of more advanced user customisation options, allowing users to personalise their experience based on preferences beyond the initial character selection. This could involve more dynamic recommendations for events based on user interactions and previous interactions.

7.1 Integration with external calendars

A potential future enhancement for the application is the integration with external calendar platforms such as Google Calendar and/or Apple Calendar, fulfilling requirements 6 & 7 (section 3.2.1). By allowing users to export or sync their registered events with their personal calendars, the application could offer a more seamless experience and better fit into users' daily routines. This functionality would help users stay organised by being able to enable automatic reminders and get the visibility of upcoming events as well as their other commitments. This kind of integration could increase the likelihood of event attendance, as users would not need to manually copy event details into separate apps. Implementing a calendar synchronisation feature could enhance the apps practicality as well as convenience, which would be an improvement for time management. Being able to follow the characters to the event would be ideal when new to the city. Other than that, there are different sections to the structural part of the application that we do not create, e.g., a database for profiles and different events that can be updated, and let venues put in their events, the user can go to. These different functionalities would be implemented if the application were coded fully for optimal user interaction and functionality. One participant in the usability test also mentioned making it possible for users of the application to see on the event how many people from the application is coming to an event (Appendix 2), this would be ideal when completely new to the city, so the users know if people from the same application is at the event. That would possibly incentivise going to the event.

7.2 Implementation of a ratings and feedback systems

To enhance user engagement and maintain ongoing improvement of the quality of the events, a feedback system could be implemented. After attending an event, users would be given the opportunity to rate their experience and leave a comment. This feature would be beneficial to both users and organisers, as well as creators who could use the feedback to identify what

worked well and what could be improved, leading to higher-quality events in the future. At the same time, new participants would be able to view these ratings and reviews before signing up, helping them make more informed decisions about which events to attend. This kind of transparency would help build more trust within the user community. The system could include something like a star rating scale, predefined review tags, and optional written comments to make sure it is intuitive and easy to use as well as collect useful feedback. In future iterations, this matching process could be enhanced with a starting questionnaire that helps guide users to a recommended character (See Table 5 on Design Additions). With the different aspects of the discussion and future work in mind, we will conclude on our problem statement.

8 Conclusion

In this project, we designed and developed an interactive prototype of a mobile application intended to answer the question:

“How can an interactive system be designed to support students in discovering local events related to their interests?”

Our problem domain consists of social concepts that are difficult to define concretely and observe. The concepts of human relations and personal interests cannot easily be quantified on the level of an individual without losing relevant details, such as social norms and individual motivations for attending events. Therefore, we focused on presenting events in a way where the user can choose what category of events best match their interests, using the categories as a tool for matching the user with the right events.

Our participatory design process played an important role in shaping this approach. By involving users throughout, from early workshops to usability testing, we developed a concept rooted in real experiences and feedback. Conducting the focus group, workshop, and usability test with the same group of international and Danish students created a coherent and consistent development process. From this process, we learned what worked and what did not work. We should have scheduled more time between the initial interview and the workshop to effectively apply early insights to the structure of our workshop. Additionally, the workshop could have been structured to include more of direct user input and creative exploration, rather than relying too much on discussion. Using the same group gave us valuable insight into user expectations and pain points.

In our usability test, we conducted an expert evaluation using Nielsen-Molich’s 10 usability heuristics. This allowed us to assess the application’s overall structure and design. While we remain aware that social behaviours and personal interests evolve, and by relying on stereotypes, the long-term relevance can be limited. Recognising the project’s limitations helped motivate us to create something that could be useful in a real-world scenario. The usability test gave us insight into how our characters were perceived and how users interpret the purpose of the application, primarily through the interface. The test revealed areas of improvement for future usability tests. The variation in the note-taking quality and level of detail impacted the consistency of our findings. In hindsight, assigning two people to the

session, one to guide and one to document, along with a more structured note-taking format, would have strengthened our evaluation. Our usability test taught us that the character personas should play a supporting role, rather than dominate the experience. We discovered which functionalities, like the exit button, were effectively communicated and which required improvement.

During the implementation phase, we followed a T-prototype approach, focusing on core features while leaving room for expansion. Using an object-oriented programming language proved to be an effective choice. It allowed us to utilise composition between classes and interactions between objects, and allowed the application to be easily extended with additional characters, events, or venues in future iterations.

Our system development process and the resulting diagrams supported us by helping us understand and coordinate the larger structure of the application, even though we only developed specific parts of our prototype. However, certain aspects of the implementation revealed some gaps in our planning. While the structural foundation was solid, we lacked equal planning for specific behaviours. Incorporating a detailed pseudocode and state-change diagram would have been beneficial to the coding process by providing clearer guidance and improving communication.

In conclusion, this project demonstrates how user-centred, participatory design approaches can inform the development of an interactive system that supports students in discovering local events aligned with their interests. By engaging real potential users through the design process and balancing conceptual and technical considerations derived from the users, we created a prototype that is functional and based on user experience. While the project has areas for improvement, particularly in planning, documentation and evaluation structure, it provides a thoughtful design in addressing social needs when moving to a new city. Future iterations can build on the prototype's foundation, creating a potential tool able to genuinely enhance student and community connection.

9 Bibliography

10 *Usability Heuristics for User Interface Design*. (n.d.) Nielsen Norman Group. Retrieved 12 May 2025, from <https://www.nngroup.com/articles/ten-usability-heuristics/>

Aalborg University. (n.d.). *Bliv buddy*. Aalborg Universitet. Retrieved 26 May 2025, from <https://www.studerende.aau.dk/valg-undervejs-og-job/faa-erfaring-mens-du-studerer/bliv-buddy>

Aalborg University. (n.d.). *Få studie- og trivselsvejledning*. Aalborg Universitet. <https://www.studerende.aau.dk/vejledning/studie-trivselsvejledning>

Aalborg University. (n.d.). *New Students*. Aalborg University. <https://www.newstudents.aau.dk/>

Aarhus University. (n.d.). *Kodning af kvalitative data*. Retrieved 26 May 2025, from <https://metodeguiden.au.dk/kodning-af-kvalitative-data>

AI for UX: Getting Started. (n.d.). Nielsen Norman Group. Retrieved 23 May 2025, from <https://www.nngroup.com/articles/ai-ux-getting-started/>

Android LinearLayout in Kotlin | GeeksforGeeks. (n.d.). Retrieved 12 May 2025, from <https://www.geeksforgeeks.org/android-linearlayout-in-kotlin/>

AU Studypedia. (n.d.). *Ny studerende*. Retrieved 26 May 2025, from <https://studypedia.au.dk/ny-studerende>

Barnum, C. M. (2021). 8—Analyzing the findings. In C. M. Barnum (Ed.), *Usability Testing Essentials (Second Edition)* (pp. 287–319). Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-816942-1.00008-3>

Baumeister, R. F., & Leary, M. R. (1995). The need to belong: Desire for interpersonal attachments as a fundamental human motivation. *Psychological Bulletin*, 117(3), 497–529. <https://doi.org/10.1037/0033-2909.117.3.497>

Benyon, D. (2014). *Designing interactive systems: A comprehensive guide to HCI and interaction design* (Third edition). Pearson.

Boblberg. (n.d.). *Boblberg | En digital platform for fællesskaber*. Boblberg. <https://boblberg.dk/studiebobler>

Boblberg. (n.d.). *Find dit nye fællesskab på Boblberg i dag*. Boblberg. <https://boblberg.dk/hvad-er-boblberg>

Bødker, S., Dindler, C., Iversen, O. S., & Smith, R. C. (2022). *Participatory Design*. Ebook Central. <https://ebookcentral.proquest.com/lib/aalborguniv-ebooks/detail.action?pq-origsite=primo&docID=6820765>

Bruselius-Jensen, M., Sørensen, N. U., & Katznelson, N. (2023). *Fællesskaber i ungdomslivet: Når de gør godt, og når de gør ondt* (1. udgave). Aalborg Universitetsforlag.

Bylov, J. (n.d). *Rammerne for ungdomslivet skaber mistrivsel*. Aalborg Universitet. <https://www.kultur.aau.dk/rammerne-for-ungdomslivet-skaber-mistrivsel-n44838>

Class Diagram | Unified Modeling Language (UML) | GeeksforGeeks. (n.d Retrieved 26 May 2025, from <https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/>

Create dynamic lists with RecyclerView | Views. (n.d.). Android Developers. Retrieved 22 May 2025, from <https://developer.android.com/develop/ui/views/layout/recyclerview>

Dam, R. F. (2025, March 13). *The 5 Stages in the Design Thinking Process*. The Interaction Design Foundation. <https://www.interaction-design.org/literature/article/5-stages-in-the-design-thinking-process>

Difference Between View and ViewGroup in Android | GeeksforGeeks. (n.d). Retrieved 12 May 2025, from <https://www.geeksforgeeks.org/difference-between-view-and-viewgroup-in-android/>

Dot Voting: A Simple Decision-Making and Prioritizing Technique in UX. (2019). Nielsen Norman Group. <https://www.nngroup.com/articles/dot-voting/>

Duck, S. (2007). *Human Relationships* (4th ed., pp. xii–xii). SAGE Publications. <https://doi.org/10.4135/9781446214336>

Favoring Composition Over Inheritance In Java With Examples | GeeksforGeeks. (n.d <https://www.geeksforgeeks.org/favoring-composition-over-inheritance-in-java-with-examples/>

Figma. (n.d.). Figma. Retrieved 26 May 2025, from <https://www.figma.com/community/file/1129468881607079432/mobile-apps-prototyping-kit>

Future Workshop – Participedia. (n.d). <https://participedia.net/method/4796>

Genlyd. (n.d.). *GENLYD - Bliv en del af et fællesskab*. Retrieved 26 May 2025, from <https://genlydaarhus.dk/>

Harrild, T. B. (n.d.). *Alkohol i studiestarten*.

Hartkopf, B. T. (2023). *De fleste studerende er glade for at studere, men en tredjedel er i risiko for mistrivsel*. <https://eva.dk/videregaaende-uddannelse/studieliv-og-trivsel/trivsel/de-fleste-studerende-er-glade-for-at-studere-men-en-tredjedel-er-i-risiko-for-mistrivsel>

Hartson, R., & Pyla, P. (2019). *The UX Book: Agile UX Design for a Quality User Experience, Second Edition* (Second Edition). Elsevier Science and Technology Books, Inc.

HashMap (Java Platform SE 8). (n.d.). Retrieved 21 May 2025, from <https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>

Helen Sharp, Jennifer Preece, & Yvonne Rogers. (2019). *Interaction Design: Beyond Human-Computer Interaction: Vol. Fifth edition*. Wiley. <https://search.ebscohost.com/login.aspx?direct=true&db=nlebk&AN=2097340&site=ehost-live>

Interaction Design Foundation. (2016). The Interaction Design Foundation.
<https://www.interaction-design.org/literature/topics/brainstorming>

Introduction to Programming Using Java, JavaFX Edition. (n.d.). Lulu. Retrieved 26 May 2025, from <https://www.lulu.com/shop/david-eck/introduction-to-programming-using-java-javafx-edition/paperback/product-1nwwq5r.html>

Kvale, S., & Brinkmann, S. (2022). *Interview: Det kvalitative forskningsinterview som håndværk* (Bjørn Nake, Trans.; 3. udgave). Hans Reitzel.

Making Your UX Life Easier with the MoSCoW. (2015, November 18). The Interaction Design Foundation. <https://www.interaction-design.org/literature/article/making-your-ux-life-easier-with-the-moscow>

Mary Fonden. (n.d). *Ensomhed i Danmark*. MaryFonden.dk.
<https://www.maryfonden.dk/ensomhed-i-danmark/>

Mathiassen, L., Munk-Madsen, A., Nielsen, P. A., & Stage, J. (2018). *Object Oriented Analysis & Design* (2. edition). Metodica ApS.

Mcmillan, D., & Chavis, D. (1986). Sense of Community: A Definition and Theory. *ResearchGate*.
https://www.researchgate.net/publication/235356904_Sense_of_Community_A_Definition_and_Theory

Nielsen, J. (n.d.). *Severity Ratings for Usability Problems: Article by Jakob Nielsen*. Nielsen Norman Group. Retrieved 14 May 2025, from <https://www.nngroup.com/articles/how-to-rate-the-severity-of-usability-problems/>

Norman, D. (2013). *The Design of Everyday Things: Revised and Expanded Edition* (Rev. and expanded ed.). Basic Books.

Papacharissi, Z. (Ed.). (2011). *A networked self: Identity, community, and culture on social network sites*. Routledge, Taylor & Francis Group. <https://doi.org/10.4324/9780203876527>

Sundhedsstyrelsen. (2022). *Mental sundhed*. <http://www.sst.dk/da/Fagperson/Forebyggelse-og-tvaergaende-indsatser/Mental-og-digital-sundhed/Mental-sundhed>

The Role of Color in UX | Toptal®. (n.d.). Toptal Design Blog. Retrieved 23 May 2025, from <https://www.toptal.com/designers/ux/color-in-ux>

Venkatakrishnan, N. (2021, September 3). The importance of asking questions in design thinking. *Bootcamp*. <https://medium.com/design-bootcamp/the-importance-of-asking-questions-in-design-thinking-ab2a5eefba20>

What is a Prototype? (n.d.). The Interaction Design Foundation. Retrieved 23 May 2025, from <https://www.interaction-design.org/literature/topics/prototypes>

10 List of Figures

Figure 1 Design Thinking Process model of our iterative design.....	13
Figure 2 Brainstorm	16
Figure 3 5W1H method	17
Figure 4: Problem areas using 5W1H and dot voting.....	18
Figure 5 Post-it notes as a result of our workshop.....	21
Figure 6 Tipsy's different design iterations	30
Figure 7 Design iterations of the characters	31
Figure 8 Paper sketch of characters	32
Figure 9 Digital coloured sketch of characters	33
Figure 10 First iteration of high-fidelity prototype.....	34
Figure 11 Comparison of first and second iteration prototype 1.A-1.C to 2.A-2C	48
Figure 12 Comparison of first and second iteration prototype 1.D-1.F to 2.D-2F	49
Figure 13 Comparison of first and second iteration prototype 1.G-1.I to 2.G-2I.....	50
Figure 14 Class Diagram that shows how the different classes in our model and function	54
Figure 15 layered architecture pattern of our application	56
Figure 16 Flowchart of user choosing a category	58
Figure 17 code snippet showing "Has-A" relationship.....	61
Figure 18 Final Application Prototype	62
Figure 19 Detailed project folder structure	64
Figure 20 UML Class Diagram of the Characters Class.....	65
Figure 21 Method for Adding Events and Linking Venues in Characters	66
Figure 22 Data structure of the FakeDatabase	66
Figure 23 Creating views dynamically in onCreate()	67
Figure 24 createCharacterView() LinearLayout setup	67
Figure 25 createCharacterView() ImageView	68
Figure 26 createCharacterView() TextView	68
Figure 27 Character click intent setup	68
Figure 28 Retrieving the Selected Character from Intent and Matching by Name	69
Figure 29 Character Navigation Using Left and Right Arrows	70
Figure 30 Connecting UI Components Using findViewById()	71
Figure 31 Accessing Elements Inside an Included Event Bubble	71
Figure 32 Retrieving the Selected Character Name via Intent	71
Figure 33 Finding the Selected Character in the List	72
Figure 34 Accessing the Selected Character Object by Index	72
Figure 35 Retrieving and Adjusting the Character's Theme Colour	72
Figure 36 Styling the Background with a GradientDrawable.....	72
Figure 37 Matching and Displaying Events for the Selected Character.....	73
Figure 38 avigating Between Characters Using Arrow Buttons38.....	74
Figure 39 Handling Category Tab Clicks with a Shared Listener	74
Figure 40 Shortcut Icons for Calendar and Home Navigation	75

Figure 41 Clickable button and Intent for Navigating to Event Details	76
Figure 42 Retrieving and Displaying Event Information in EventDetailsActivity.....	77
Figure 43 Exit Button Implementation Using finish()	77
Figure 44 Setting Initial Theme and Loading Events in onCreate()	78
Figure 45 Matching Character Events with loadCharacterEvents()	78
Figure 46 Converting Event Dates with convertToDateKey()	79
Figure 47 Identifying Calendar Days with Events in updateCalendar().....	80
Figure 48 Populating Calendar Grid and Assigning Adapter in updateCalender()	80
Figure 49 ViewHolder Class with RecyclerView.....	81
Figure 50 Handling Click Events on Calendar Dates Using ViewHolder.....	82
Figure 51 Populating Calendar Dates in onBindViewHolder()	82